CONEX OPERATIONS

showers.tex

CONEX OPERATIONS

Hybrid simulations of
extensive air showers

```
$ What is there to explore in this notebook?
> install CONEX
> simulate showers
> win the Nobel Prize. ■
```

01 / ABOUT THIS NOTEBOOK

This guide takes you from installing CONEX to your first simulations. I explore basic settings and the meaning of some more advanced flags. I hope it becomes a useful first companion for simulating particle showers.

"As long as I have questions and no answers, I will keep writing."

00 / START HERE Guide / APOEMA

Document map

Topics and pages

OBJECTIVE Find a topic and jump to its first page.

1 Reading path: How to use this guide 3

2 Fundamentals: What is being simulated? 5

3 Environment: Before installation 7

4 Installation: Install CONEX 14

5 First run: A verifiable shower 17

6 Configuration: Energy and sampling 23

7 Python analysis: Read results with PyROOT 29

8 C++ analysis: Read results with C++ 38

9 Routine: Everyday use 47

10 Reference: Linked glossary 50

11 Sources: References and further reading 53

12 Reference: Linux commands 55

This compact contents page lists topics. Internal subsections remain numbered on the content pages.

01 / READING PATH
Guide / APOEMA

How to use this guide

Every operation should have a meaning

OBJECTIVE Choose a reading path and recognize the verification level of each procedure.

This guide takes you from preparing the environment to simulating and comparing extensive air showers. Read the fundamentals before running commands, then complete the first simulation and choose one analysis route. The terminal examples use Bash on Linux, including Linux inside WSL 2. Commands for PowerShell are explicitly identified. Do not copy prompt symbols or visual line wrapping from the PDF.

1.1 Where the content came from

This guide grew out of work by students in the APOEMA group after years of using CONEX. I particularly thank Marcos Olegário, who contributed the installation route and the initial explanation of the output trees.

The material began when I was still learning to simulate showers with CONEX. The lack of readily accessible documentation, compared with CORSIKA, makes CONEX harder to get started with, even though its approach is simpler.

I have therefore collected the main practices and settings so that anyone starting air-shower simulations with CONEX can follow a coherent route, without years of frustrating searches. I hope it proves useful.

1.2 Distribution covered here

> NOTE /
>
> This document covers the CONEX 7.8 distribution studied for this guide. It may no longer be current when you read it.

1.3 Reading routes

Beginners: read sections 2 to 6 in order, then choose **one** route: Python with PyROOT, section 7, or C++ macros, section 8. Both teach opening files, reading trees and branches, plotting profiles and histograms, comparing samples, and examining leading interactions. You do not need to read or run both.

If you already compile CONEX, start at section 5 and consult section 6 as needed. After your chosen route, continue to Everyday use, section 9. The glossary and command reference are at the end; their internal links work offline.

SOURCE / README, Makefile, src/conex_sub.F:1053 and src/CxRoot.cc:335; public cxroot tags [5].

01 / READING PATH [continued]
Guide / APOEMA

How to use this guide

Download the companion files

OBJECTIVE Obtain the code and keep the directory structure expected by the commands.

The examples are in davisilva169/CONEX-guide on GitHub. Open the link and choose **Code** > **Download ZIP**. Extract it into a Linux or WSL working directory, preferably without spaces in the path. You do not need a GitHub account to download it.

1.4 What to download

- `python`: the Python script implementing the analysis operations.
- `macros`: the five C++/ROOT alternatives. Download the whole directory: `histogram.C` uses `compare.C`.
- `scripts`: small-campaign generation and example reproduction.
- `results`: recipes for generating ROOT files, reference plots, a summary and provenance.

The repository provides this guide as English and Brazilian Portuguese PDFs. The editable LaTeX sources are supplied separately. Obtain and install CONEX, its tables and the dependencies through the following sections.

1.5 Where to run commands

When the text says *repository root*, enter the extracted directory containing `python`, `macros` and `scripts`. Adjust this example to your chosen location:

enter the example directory — bash / code 1.1

```bash
cd "$HOME/path/to/CONEX-guide-main"
ls python macros scripts results
```

Keep this structure. To download an individual file through a guide link, open its GitHub page and use **Download raw file**, saving it in the indicated directory. Starting with the complete ZIP avoids missing a dependency.

> **NOTE / Download code; generate event data**
>
> Links beside command blocks lead to the required code. ROOT filenames link to their recipes in `results`, not to event downloads. First produce the output in your installation, then set `ARQ`, `A`, `B` or `LIDER` to its actual path. The reference plots and summary can be downloaded for comparison.

02 / FUNDAMENTALS Guide / APOEMA

What is being simulated?

The cascade and the hybrid approximation

OBJECTIVE Connect the computational method to the quantities in the output.

CONEX is a hybrid simulation program combining the Monte Carlo method and cascade equations to simulate extensive air showers.

A primary cosmic ray can interact with air nuclei and produce secondary particles. Further interactions and decays develop an extensive air shower. CONEX calculates profiles along the shower axis, including event-to-event fluctuations.

At higher energies, Monte Carlo samples individual processes. Particles below the thresholds feed source terms for cascade equations: a numerical description of particle populations in energy and depth. This combination makes it cheaper to study profiles and energy deposit than to track every particle explicitly. Bergmann et al. describe the algorithm and its comparison with full simulations [1]; the initial paper by Pierog et al. documents the motivation and early results [2].

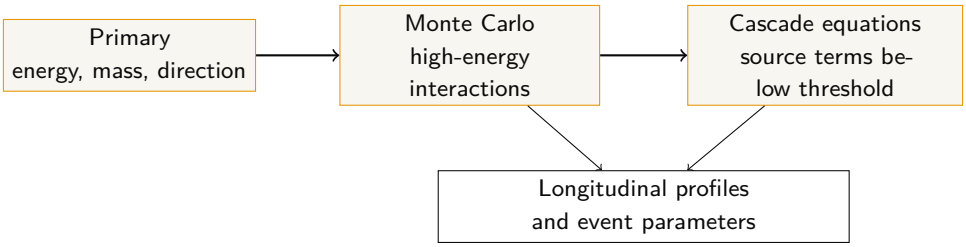


Figure 2.1: Schematic representation of the processes involved in a CONEX simulation.

> **CONCEPT 2.1 / Interaction model**
>
> A physical prescription for collision probabilities and secondary-particle production. Choosing a model changes the physics; it is more than a speed setting.

2.1 The scope of the 1D description

The main coordinate is depth along the axis. The standard CONEX mode studied here does not provide a complete lateral particle distribution for a surface detector. Profiles and fluctuations help with composition studies and fluorescence techniques, but do not replace optical propagation, detector response or experimental reconstruction. This is a real limitation that should be clear from the outset.

The code contains CORSIKA integrations and three-dimensional options, but the standalone `bin/conex2r` executable and the CMake integration with CORSIKA 8 serve different purposes. Do not enable compilation macros to obtain 3D as though they were an execution flag validated by this guide.

SOURCE / README: INTRODUCTION; src/conex_sub.F preprocessor options; CMakeLists.txt; [1, 2].

02 / FUNDAMENTALS [continued] Guide / APOEMA

What is being simulated?

Quantities and units before flags

OBJECTIVE Interpret a profile without confusing energy, particle count, altitude and depth.

CONCEPT 2.2 / Primary

The particle initiating the shower. Its type, total energy and direction define the initial conditions; events with the same conditions can still fluctuate.

Quantity	Meaning and unit
E	Total primary energy; flags receive $\lg E = \log_{10}(E/\mathrm{eV})$, not E in GeV.
θ	Zenith angle in degrees; zero is the vertical downward case used here.
X	Atmospheric depth along the axis (slant depth), in $\mathrm{g\,cm^{-2}}$.
H	Altitude in metres, distinct from X.
$N(X)$	Number of charged particles above the specified cuts. Photons are not charged particles.
dE/dX	Energy deposited per depth: $\mathrm{GeV/(g\,cm^{-2})}$.
$X_{\max}$	Depth of shower maximum, estimated from a Gaisser–Hillas fit to the charged-particle profile, in $\mathrm{g\,cm^{-2}}$.

CONCEPT 2.3 / Atmospheric depth

Mass of air per area traversed along a trajectory: $X(s) = \int \rho(s')\,ds'$. For a specified atmosphere, H and X are related, but are different coordinates.

Altitude describes geometric position; depth describes traversed matter. In the flat-atmosphere approximation, $X_{\mathrm{slant}} \simeq X_{\mathrm{vertical}}/\cos\theta$ at moderate angles. Near the horizon, curvature and geometry matter: use the program's coordinates rather than this approximation to replace the atmosphere.

EXAMPLE 2.1 / Read the energy scale

`-e 17 -E 17` fixes $E = 10^{17}$ eV $= 10^{8}$ GeV. The value 19.5 corresponds to about 3.16×10^{19} eV. Use a decimal point in commands. The program converts eV to GeV internally in `GetEnergy`.

EXERCISE 2.1 / Practice, without solutions

1. Convert 10^{18} eV to GeV and explain the difference between entering 18 and 10^{18} in an energy flag.
2. Two showers have the same H and different angles. Why does that not fix the same X?
3. Why does a single profile not represent the fluctuations of a shower sample?

SOURCE / README branch descriptions; src/CxRoot.cc:GetEnergy; physical definition of depth and the method in [1].

03 / ENVIRONMENT Guide / APOEMA

Before installation

Terminal, paths and tools

OBJECTIVE Know where a command runs and where its tools come from.

Throughout the guide we use terminal commands to navigate directories, check files and run programs. The Linux command table at the end collects these instructions and examples, including `ls`, `cd`, `cd ..` and `mkdir`. Consult it whenever you need a reminder.

Obtain the CONEX code and companion files directly from the IAP server. Access is password protected: open `https://web.iap.kit.edu/corsika/download/` in your browser.

When asked to authenticate, enter the username `corsika` and the password received by email after requesting access from the collaboration. Open the `conex/` directory listed there and select the packages you need.

> **CONCEPT 3.1 / Terminal and shell**
>
> The terminal is the text interface. The shell interprets the commands. This guide uses Bash on Linux; Windows PowerShell is a different shell with different syntax.

> **CONCEPT 3.2 / Paths**
>
> An absolute path starts at the filesystem root, for example `/home/ana/conex`. A relative path, such as `cfg/conex_sibyll.param`, depends on your current directory. A path is the sequence of characters locating a file or directory in a filesystem. glossary

> **NOTE /**
>
> Linux filenames are case sensitive.

03 / ENVIRONMENT [continued] Guide / APOEMA

Before installation

Inspect the environment

OBJECTIVE Check the system and tools before choosing an installation route.

Open a Linux terminal. Diagnose the environment before installing anything. Run the commands below; a command can fail if its tool is not yet installed or activated.

inspect the environment bash / code 3.1

```bash
pwd # Show the current directory.
uname -a # Kernel, system and architecture.
cat /etc/os-release # Linux distribution and version.
command -v gcc g++ gfortran make root root-config
gcc --version
gfortran --version
root-config --version
root-config --cflags --libs
```

This step identifies the system and available tools, versions and paths. That information helps you choose compatible dependencies and recognize what still needs installation. A missing command can mean that the tool is absent or that its environment needs activation. The final line reports the compiler and linker options required by this ROOT installation.

> CONCEPT 3.3 / Compiler
>
> Translates source code to executable code. This package combines C, C++ and Fortran; a C++ compiler alone does not provide all three languages.

> CONCEPT 3.4 / Library
>
> A collection of functions used by other programs. Model libraries are loaded at runtime; compiling the executable does not prove that a model can be loaded and initialized.

03 / ENVIRONMENT [continued] Guide / APOEMA

Before installation

Where each component fits

OBJECTIVE Recognize the source project, generated files and actual dependencies.

Component	Role in this distribution
README	Installation, options, output and references.
Makefile	Builds the standalone executable and models.
src/	C++ interface, Fortran physics, included models, paramin files and headers.
tabs/	Physics tables, EGS4, UrQMD, models and compressed-data reader.
macros/	The authors' reading and plotting examples; some use older names.
resampler/	Extensions modifying interactions, outside the introductory route.
cfg/	Generated during compilation: param files with substituted paths.
obj/Linux/	Intermediate compilation objects.
lib/Linux/	Model and interface libraries.
bin/	The conex2r executable.

> **CONCEPT 3.5 / Makefile**
>
> A recipe of dependencies and compilation commands read by Make. `make sibyll` builds a target; `-m 5` selects the model in a later simulation.

The ROOT-output executable requires C/C++/Fortran compilers, Make, Unix utilities, compatible ROOT and bzip2 development files (header and library). The Makefile requests C++17 and links `-lbz2`. Python, Jupyter, Conda and Anaconda are not intrinsic CONEX requirements.

`CX_NO_ROOT=1` is a variable recognized by the Makefile in the CONEX root. Passing it as `make sibyll CX_NO_ROOT=1` disables the ROOT route and leaves a library build for integration. This is not the installation used to generate and analyse the files in this guide.

> **NOTE /**
>
> CMakeLists.txt refers to external CORSIKA 8, Sibyll and UrQMD targets. Do not assume `cmake ..` replaces the standalone Makefile.

3.1 Paths containing spaces

Quotes protect spaces in Bash, but this package's Fortran truncates certain filenames at the first space, and the Makefile expands some paths without sufficient protection. Keep the *build copy and tables* in a short Linux path without spaces. Avoid special characters and accents as well.

SOURCE / File paths: src/conex2r.F and Makefile. Package structure: README, Makefile and CMakeLists.txt.

03 / ENVIRONMENT [continued] Guide / APOEMA

Before installation

Linux: choose a consistent environment

OBJECTIVE Install dependencies without accidentally mixing incompatible versions.

> CONCEPT 3.6 / ROOT
>
> CERN tools for scientific data: files, trees, histograms and plots. ROOT includes C++ libraries and an interpreter; PyROOT provides its Python interface.

We distinguish two ways to obtain ROOT: install a distribution directly on the system, or use an Anaconda/Conda environment. The first is presented only as an alternative; we then follow the second, which installs ROOT with one command. On Windows, first prepare Linux through WSL 2, then return here.

The main CONEX route is Linux with ROOT and compilers from a consistent environment, either maintained by your group or isolated. For Ubuntu/Debian, the recipe below documents the direct-installation alternative. Its first two lines install compilers and libraries, *not ROOT*. The remaining lines apply only after obtaining ROOT from the official documentation and replacing `/path/to/root` with its actual location.

system dependencies - alternative route bash / code 3.2

```bash
sudo apt update
sudo apt install build-essential gfortran libbz2-dev
# Install ROOT following its official platform instructions.
# Match the binary to the system, architecture and GCC version.
source /path/to/root/bin/thisroot.sh
root-config --version
```

We do not develop direct ROOT installation further here. Consult the dependencies and binaries for your distribution, architecture and compiler [8, 9]. To follow the route developed next, continue with Anaconda; you do not need another manually installed ROOT first. Conda activation prepares ROOT on that route: do not source `thisroot.sh` from a different installation.

SOURCE / ROOT: official installation instructions and dependencies [8, 9].

03 / ENVIRONMENT [continued] Guide / APOEMA

Before installation

Second alternative: Anaconda

OBJECTIVE Install the manager used to obtain ROOT with a few commands.

> **CONCEPT 3.7 / Anaconda, Conda and environments**
>
> Anaconda is a distribution containing Python, Conda and scientific tools. Conda installs packages and creates separate environments: directories with their own program and library versions. Here we use it to obtain ROOT and PyROOT.

We follow Anaconda because Conda can obtain ROOT and its dependencies after the environment is prepared. This simplifies setup, especially for beginners. Python is the language used through PyROOT; Anaconda is a convenience, not a CONEX requirement.

3.2 Obtain and open the installer

1. Visit anaconda.com/download and find *Anaconda Distribution*.
2. Choose the **Linux** installer for the architecture reported by `uname -a`. Choose Linux in WSL too: CONEX will run inside Ubuntu.
3. Save the `.sh` file. Rename it to `anaconda-installer.sh` in your file manager to use the short name below.
4. Open a Linux terminal in the download directory. If downloaded through Windows, check the actual location, for example:

   `/mnt/c/Users/your_user/Downloads`

open the installer from its download directory — bash / code 3.3

```bash
bash ./anaconda-installer.sh
```

Follow the licence and destination instructions displayed in the terminal. When asked whether to initialize Conda in your shell, choose `yes`, so later sessions recognize `conda`. Close and reopen the Linux terminal after installation.

3.3 Check the installation

Run:

check Conda — bash / code 3.4

```bash
conda --version
```

Expect a Conda version. If the command is not found, check that initialization completed and the terminal was reopened. For the default destination, you can also run `source "$HOME/anaconda3/etc/profile.d/conda.sh"` in the current session; adapt the path if you chose another destination.

SOURCE / Anaconda: Linux installer; Conda environments [14]; ROOT through Conda [8].

03 / ENVIRONMENT [continued] Guide / APOEMA

Before installation

Create the environment and install ROOT

OBJECTIVE Prepare the tools in steps and check ROOT and PyROOT in the same environment.

3.4 1. Create a separate environment

Create conex-apoema in the terminal where Conda worked:

create the environment bash / code 3.5

```
conda create -n conex-apoema
```

Review the proposed changes and confirm when prompted.

3.5 2. Activate the environment

Activate it. The second line sets strict channel priority for this environment to reduce channel conflicts:

activate and configure the environment bash / code 3.6

```
conda activate conex-apoema
conda config --env --set channel_priority strict
```

Check that (conex-apoema) appears in the prompt, and keep using this session.

3.6 3. Install ROOT

Install from conda-forge:

install ROOT and PyROOT bash / code 3.7

```
conda install -c conda-forge root
```

Confirm the packages and wait for completion. This installs ROOT and its Python interface, PyROOT.

3.7 4. Check both interfaces

Without leaving the environment, run:

check ROOT and Python access bash / code 3.8

```
root-config --version
python -c "import ROOT; print(ROOT.gROOT.GetVersion())"
```

Both should report the ROOT version; the second confirms `import ROOT`. Activate again with `conda activate conex-apoema` in every new session.

Before installation

Windows and alternatives

OBJECTIVE Distinguish Linux on Windows from a native CONEX port.

3.8 WSL 2: the Windows route

On Windows, prepare Ubuntu before the Anaconda and CONEX commands. Perform this step on Windows, then work in the Linux terminal. WSL 2 runs a Linux distribution using a Linux kernel; the CONEX instructions remain Linux instructions.

If WSL is absent, Microsoft documents opening PowerShell as administrator and installing a distribution; restarting may be required [15]. Run `wsl -install -d Ubuntu` only if WSL/Ubuntu is not installed. Restart if requested and create your Linux user. Then use the last two lines to list distributions and open Ubuntu:

PowerShell - preparation recipe / code 3.9

```
wsl --install -d Ubuntu
wsl --list --verbose
wsl -d Ubuntu
```

Continue the Linux route in Ubuntu. A Windows path such as `C:\Users \ana \Downloads` normally appears as `/mnt/c/Users/ana/Downloads`. Prefer the Linux filesystem for compilation and table access, and avoid spaces in the CONEX copy's path.

3.9 Native installation: limited viability

ROOT has a Windows distribution, but that does not establish native support for this package. The Makefile recognizes Linux and Darwin, defines `.so`/`.dylib` libraries, and the interface uses `dlopen` and `dlfcn.h`. It references `/dev/null`, `/dev/urandom` and Unix tools. MinGW compilers do not automatically provide a combination compatible with Visual Studio ROOT. A port would need changes to dynamic loading, build recipes and Fortran/C++ interoperability, followed by physics tests. We therefore use Ubuntu in WSL 2 rather than attempt a native Windows CONEX build.

3.10 Other relevant routes

A Linux virtual machine is an alternative where WSL is not allowed; it requires disk space, memory and its own installation. A group Linux server can also compile and analyse, transferring only small results. Start with Linux or WSL 2; leave containers and macOS adaptations to dedicated installation instructions.

SOURCE / Microsoft WSL documentation [15]; standalone distribution README and Makefile.

04 / INSTALLATION Guide / APOEMA

Install CONEX

Prepare the working copy

OBJECTIVE Gather the files and tools before compiling the simulator.

With ROOT working, prepare CONEX. Keep the session with `conex-apoema` active. If you chose direct system installation, use the session where ROOT and the system compilers are available.

4.1 Build tools

For the Conda route, install C/C++ and Fortran tools, Make and bzip2 now. These compile CONEX and are separate from installing ROOT. Reactivate the environment afterwards to load compiler settings:

build tools in the active environment bash / code 4.1

```
conda install -c conda-forge cxx-compiler fortran-compiler make bzip2
conda deactivate
conda activate conex-apoema
```

Check that the commands below report compiler versions and a Make path. The activation packages define `CXX` and `FC`, identifying the compilers:

check environment tools bash / code 4.2

```
"$CXX" --version
"$FC" --version
command -v make
```

If one fails, repair the installation or reactivate the environment before proceeding. For the direct system route, check the system's `g++`, `gfortran` and `make`.

4.2 Organize the downloaded files

Extract the CONEX package obtained at the start of section 3. Locate `Makefile`, `README`, `src/` and `tabs/`; that directory is the root to use, even if extraction produced repeated directory names.

Using your file manager, create `simulations` inside your Linux home directory. Copy the complete CONEX root there and name the copy `conex-7.801`. This lets you follow the commands without changing the downloaded material. If you choose another path, adjust the variable below. The directory name is only a working convention, not a claim that folder, internal and output-format versions are identical.

> **CONCEPT 4.1 / Environment variable and PATH**
>
> An environment variable is a value passed by the shell to programs, such as an output directory. PATH lists directories searched for executables; it is not the list of CONEX tables.

SOURCE / Dependencies and directories: Makefile and README; Conda compiler packages.

Install CONEX

Compile and check the installation

OBJECTIVE Build the executable and SIBYLL library for the first simulation.

Start with SIBYLL, the model used next. This prepares only what the first run needs. Execute in the Linux terminal with the environment active:

enter the copy and compile SIBYLL — bash / code 4.3

```bash
CX="$HOME/simulations/conex-7.801"
cd "$CX"
ls Makefile README src tabs
root-config --version
make sibyll CXX="$CXX" CC="$CC" FC="$FC" > build.log 2>&1
```

`CX` is our shortcut for the copy's path, not a CONEX option. Bash expands `$CX` automatically inside double quotes; do not replace it manually after assigning the variable. The Make line compiles and saves messages in `build.log`. Wait for the prompt to return. For system compilers, replace only that line with `make sibyll > build.log 2>&1`.

Check the generated files and display help:

check executable, library and help — bash / code 4.4

```bash
ls -lh bin/conex2r lib/Linux/libCONEXsibyll.so
bin/conex2r -h
```

Expect the executable and library, and help listing the program's options, including model 5, SIBYLL. If compilation fails or a file is missing, read the first error in `build.log` before continuing. Help can exit with a nonzero status even after being displayed; here, check its text.

`make` without a target displays build help. `make all` prepares all models, but is unnecessary for the first activity. Compilation prepares the program; the next section checks it with an actual shower.

SOURCE / Makefile: sibyll, binary and dynamic targets; executable help.

Install CONEX

Configuration and output paths

OBJECTIVE Know where CONEX looks for files and writes results.

Next we use CONEX_ROOT for configuration files and ROOT_OUT for output. This is a reference table; you do not need to set every variable for the first run.

Variable	Role
CONEX_ROOT	Directory of param files. If absent, the interface looks in PWD/cfg.
ROOT_OUT	ROOT output directory. Defaults to PWD if absent; it must exist.
CORSIKA_DATA	Selects the table root at compilation, before CONEXTABROOT.
CONEXTABROOT	Alternative table root; the default uses the installation's tabs directory.
CONEXBINROOT	Alternative build prefix in this Makefile. Initially keep the files in the copy.
URQMD13_TAB	UrQMD decay-width table name; defaults to tables.dat.

4.3 Check paths before the first run

Still in CX, run:

check the generated configuration bash / code 4.5

```
ls cfg/conex_sibyll.param
head -n 20 cfg/conex_sibyll.param
```

The first command checks that the param file was created; the second shows its beginning. Table paths should point to the copy or your chosen data directory. They were written during compilation; moving the project can require regenerating them.

URQMD13_TAB concerns the decay-width table tables.dat, which may be generated during initialization. This differs from the cross-section table UrQMD-1.3.1-xs.dat referenced by furqmd dat in the param file. Keep them distinct when diagnosing missing files.

> **NOTE / Before continuing**
>
> Use the session where ROOT is active, keep CX pointing to the compiled copy and check the param file. Section 5 starts from this state. In new sessions, reactivate the environment and redefine CX. Avoid make clean in shared installations: work in your own copy.

SOURCE / Makefile; README; CxRoot.cc runtime interface.

05 / FIRST RUN Guide / APOEMA

A verifiable shower

Understand a simple execution

OBJECTIVE Understand a simple command and produce a first result that can be checked.

A CONEX run starts by choosing the shower conditions: the primary particle, its energy and arrival direction, and the model describing its high-energy hadronic interactions. We also specify how many showers to simulate.

Start with **one proton at 10^{17} eV, arriving vertically, using SIBYLL 2.3e**. Holding these conditions fixed makes it easier to compare the request with the output.

This section introduces only the options needed for this first example. The next section details flags, defaults, other particles, available models and further settings.

5.1 How the command is organized

Call `bin/conex2r`, followed by the options defining the run. A *flag* is an option such as `-n`; its following value specifies the choice, as in `-n 1`.

For our first example:

Option	What it selects
`-p 100`	Primary particle: a proton.
`-m 5`	High-energy hadronic model: SIBYLL 2.3e in this distribution.
`-n 1`	Number of showers: just one.
`-e 17 -E 17`	Equal lower and upper energy limits, giving fixed energy.
`-z 0 -Z 0`	Equal zenith-angle limits of zero degrees, giving vertical incidence.
`-s 314159`	Seed of the pseudorandom generator.
`-x first`	Prefix included in the output filename.

5.2 How to specify energy

`-e` and `-E` receive the decimal logarithm of the energy in eV:

$$\ell = \log_{10}\left(\frac{E}{\mathrm{eV}}\right), \qquad E = 10^{\ell}\,\mathrm{eV}.$$

Thus `-e 17 -E 17` means $E = 10^{17}$ eV, not 17 eV. Equal limits give the same initial energy to every requested event. For a nucleus, this is total primary energy, not energy per nucleon.

5.3 How to specify a nucleus

For the nuclear primaries considered here, CONEX uses

$$\text{primary code} = 100 \cdot A,$$

where A is the mass number: the total number of protons and neutrons.

A proton is 100; helium-4, 400; carbon-12, 1200; and iron-56, 5600. We use `-p 100`. Do not apply this rule to non-nuclear particles, discussed next.

SOURCE / Studied distribution: README and CxRoot.cc runtime option parsing.

05 / FIRST RUN [continued] Guide / APOEMA

A verifiable shower

Run the first shower

OBJECTIVE Execute the simulation and recognize its output files.

The following block runs your first CONEX simulation and places the results in a dedicated directory. Execute it in the Linux terminal, with ROOT available, the environment active and `CX` pointing to the compiled copy.

> **CONCEPT 5.1 / Seed**
>
> A value initializing the pseudorandom generator that supplies simulation draws. Recording it helps reproduce a run, but using the same value does not guarantee identical results across versions, models or environments.

Run the lines in order. If a command fails, stop and investigate before continuing. If `first-run` already exists, choose another name and change its occurrences below.

first execution bash / code 5.1

```bash
cd "$CX"
export CONEX_ROOT="$CX/cfg"
mkdir first-run
export ROOT_OUT="$CX/first-run"
bin/conex2r -m 5 -p 100 -e 17 -E 17 -z 0 -Z 0 \
  -n 1 -s 314159 -x first > first-run/run.log 2>&1
status=$?
printf 'Exit status: %s\n' "$status"
ls -lh first-run
tail -n 25 first-run/run.log
```

First enter the CONEX directory and set `CONEX_ROOT` to its configurations. Create `first-run` and select it as the ROOT output directory with `ROOT_OUT`. These instructions organize the run; they do not change energy or particle type.

The `bin/conex2r` line runs the shower with the choices on the previous page. The final backslash continues the same command on the next line. `> first-run/run.log 2>&1` sends normal and error messages to one text file, `run.log`; the simulation can therefore run without showing progress in the terminal.

Immediately afterwards, `status=$?` stores the exit status, and `printf` displays it. Zero normally indicates success. `ls -lh` lists output files and sizes; `tail -n 25` displays the final 25 log lines so you can check completion.

> **NOTE / Avoid overwriting**
>
> `ROOT_OUT` does not create a directory, hence `mkdir` comes first. Reusing prefix, model, seed and particle in the same directory can replace a previous file, even if energy or angle changed. Use a new directory for each run you want to keep.

SOURCE / CxRoot.cc: option parsing, ROOT_OUT and output filename construction.

05 / FIRST RUN [continued] Guide / APOEMA

A verifiable shower

Check what was produced

OBJECTIVE Check that the file opens and its contents match the requested run.

For this example, the expected filename is

`first_sibyll23e_000314159_100.root`

It combines the prefix `first`, model identifier `sibyll23e`, zero-padded seed and primary code 100. Check the actual filename in your directory. Its link leads to the recipe in `results`; generate the event file locally, since events are not distributed in the repository.

5.4 What counts as a successful run?

Finding a `.root` file is not enough: it could have been created before an interruption. Check three things:

1. The log ends without a configuration, library or table failure.
2. ROOT opens the file and finds `Header` and `Shower` trees.
3. `Shower` has one entry, matching the requested proton, energy and direction.

A ROOT tree organizes data into entries and fields. For now, `Header` contains run information and `Shower` contains shower data. The analysis section develops these objects and structures.

A verifiable shower

Open the file and plot its profile

OBJECTIVE Inspect the output structure and produce the first plots.

Use Python with PyROOT for this first check, taking advantage of the Conda environment. Section 7 explains it step by step. If you prefer C++, section 8 teaches the same operations with macros; you only need one analysis route.

Download the repository as explained on page 4 and open the Linux terminal in its extracted root, with the environment active and `CX` defined. The `python/conex.py` script is available at the link; download the full package to preserve its directories. Run:

inspect and plot the first profile bash / code 5.2

```bash
export ARQ="$CX/first-run/first_sibyll23e_000314159_100.root"
python python/conex.py inspect "$ARQ"
python python/conex.py profile "$ARQ" --output profile-python
```

On GitHub: `python/conex.py`; first-event recipe.

`ARQ` stores the generated file's absolute path; adjust it if your organization differs. The first analysis command lists keys, trees, types and the first entry. Missing `LeadingInteractions` is normal here: this run did not request `-K`.

The second command selects event zero, the first entry, and creates `profile-python.pdf` and `profile-python.csv` in the current directory. The PDF contains both plots; the CSV stores their values, leaving the final point without an associated deposit interval. Choose a new prefix if those files exist: the script refuses to overwrite them.

> **RESULT 5.1 / What to look for**
>
> Expect one entry each in Header and Shower. Check primary 100, high-energy model 5, logarithmic energy 17 and zenith zero. Profiles can fluctuate between realizations; their exact shape is not an installation-success criterion.

SOURCE / `python/conex.py`: inspect and profile; output structure in CxRoot.cc.

A verifiable shower

Interpret the first profile

OBJECTIVE Connect the plots to shower development without drawing statistical conclusions from one event.

Open your `profile-python.pdf`. Observe how the cascade develops through the atmosphere. The figure shows a reference realization with the same choices: a proton, 10^{17} eV, vertical incidence and SIBYLL 2.3e.

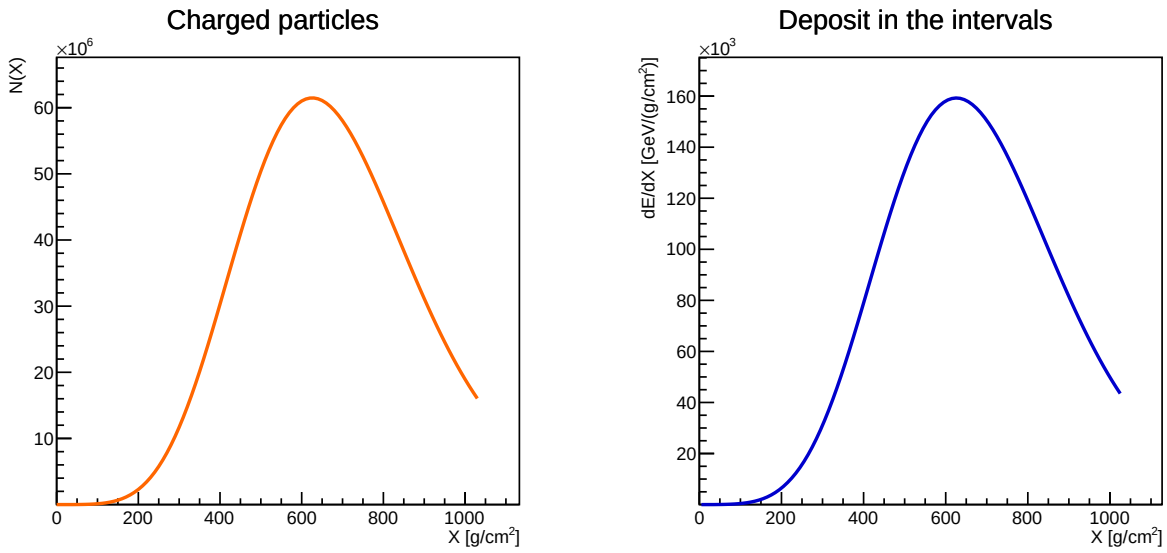


Figure 5.1: A real proton shower at 10^{17} eV, vertical, using SIBYLL 2.3e and UrQMD, seed 314159. Generated with `profile.C`. The vertical axes represent different quantities.

5.5 What are we looking at?

Both horizontal axes show atmospheric depth X, in $\mathrm{g\,cm^{-2}}$, along the shower's development: mass traversed per area, not altitude.

The charged-particle profile shows how the number varies with depth. Early interactions produce secondaries and the population grows. It reaches a maximum and then decreases as energy is shared and particles can no longer sustain multiplication.

The other plot shows energy deposited per depth, dE/dX, in $\mathrm{GeV/(g\,cm^{-2})}$. This differs from particle count, so their maxima need not occur at exactly the same depth.

A verifiable shower

A first worked result

For the reference event, summing the deposit over the 103 intervals between 104 stored points gives about 7.98×10^7 GeV. Compared with

$$10^{17}\,\mathrm{eV} = 10^{8}\,\mathrm{GeV},$$

this is approximately 80% of the primary energy.

This is the deposit integral over the intervals recorded by the macro. It is neither a detector's measured energy nor proof that the entire initial energy was deposited in that atmospheric segment.

Check whether the profile rises and falls within the recorded range. A maximum outside that range or a truncated profile requires checking geometry, ground and depth limits before interpretation.

> **NOTE / The scope of this first example**
>
> A single shower teaches execution and reading, but does not determine a mean or a fluctuation distribution. Differences between two isolated events do not establish systematic behaviour of a primary or model.

> **EXERCISE 5.1 / Practice, without solutions**
>
> 1. Check that primary, energy and direction in the file match the command. Why use equal energy limits and equal angular limits?
> 2. Run a shower with another seed in a different directory and compare the profiles. Describe a similarity and a difference without attributing statistical significance.
> 3. Using $100A$, write the code for iron-56. Which command option would change, and how would you preserve the earlier output?

SOURCE / Reference figure and values: `results/PROVENANCE.md`. Both routes integrate over the same stored intervals.

06 / CONFIGURATION Guide / APOEMA

Energy and sampling

What is a runtime argument?

OBJECTIVE Predict the sample's conditions before running the program.

A flag is an executable option, such as `-n 12`, not a compiler option. By contrast, `CX_NO_ROOT=1` changes compilation; `ROOT_OUT` is an environment variable; and `set fehcut 0.005` is an instruction in a physics configuration file. These mechanisms act at different stages and have different effects.

Syntax	Code default	Meaning, constraints and example
`-e value`	16.5	Lower limit in $\log_{10}(E/\mathrm{eV})$, e.g. -e 17.
`-E value`	21	Upper limit in the same units, e.g. -E 17.
`-a value`	3	Dimensionless sampling index, e.g. -a 0.
`-n integer`	1	Shower count: use a positive integer, e.g. -n 12.

6.1 The energy distribution matters

For $\alpha > 1$, where α is the value of `-a`, `GetEnergy` samples $dN/dE \propto E^{-\alpha}$ and rejects energies above the maximum. For $0 \leq \alpha \leq 1$, it samples uniformly in $\log_{10} E$, equivalent to $dN/dE \propto 1/E$. For $\alpha < 0$, it samples a logarithmic grid of spacing $|\alpha|$; it does not traverse the grid in sequence. Equal energy limits make the index irrelevant.

> **EXAMPLE 6.1 / Two different ways to fill an interval**
>
> `-e 17 -E 18 -a 0` gives equal sampling per logarithmic interval. `-e 17 -E 18 -a 3` favours lower energies according to a law in linear energy. The histograms answer different questions unless the sampling and, when appropriate, weights are taken into account.

6.2 Invalid input is not a protected interface

The parser uses `atoi`/`atof` with limited validation; malformed text can become zero. Use decimal points and check the printed summary. Require $e \leq E$ before calling: reversed limits can prevent energy rejection sampling from terminating. Parser acceptance does not establish physical applicability. An internal description mentions 0.1 GeV to 10^{13} GeV, but models and tables impose their own limits. This guide's tests do not validate the endpoints.

SOURCE / CxRoot.cc: constructor, GetOptions:660–755 and GetEnergy:510–539; README; Fortran ConexInit comment.

06 / CONFIGURATION [continued] Guide / APOEMA

Energy and sampling

Direction and geometry

OBJECTIVE Distinguish fixed direction from a distribution of incidence angles.

Syntax	Default	Unit, meaning and example
`-z value`	60	Minimum zenith angle, degrees; -z 0.
`-Z value`	60	Maximum zenith angle, degrees; -Z 0.
`-i value`	0	Fixed azimuth, degrees; a negative value samples $[0, 360)$, e.g. -i -1.
`-o value`	0	Minimum impact parameter, metres; -o 0.
`-O value`	0	Maximum impact parameter, metres; -O 0.

6.3 Angular sampling

The README documents $0 \leq \theta \leq 180°$: above $90°$, the shower is upward-going. For unequal limits within one hemisphere, the code implements isotropic incidence on a plane, with density proportional to $|\cos\theta|$ in the variable $\cos\theta$. This is not uniform in θ. Equal limits fix direction; set both to $90°$ for horizontal incidence.

The code stops for an interval whose lower bound is below and upper bound above $90°$. Do not mix hemispheres or reverse the limits. Our tests are vertical; horizontal and upward-going cases are documented and inspected in the code, rather than tested here.

6.4 Azimuth: state the convention

Help and README specify the Auger convention, $0°$ to the east. The code uses projections $x \propto \cos\phi$ and $y \propto \sin\phi$. Earlier material said "clockwise" without specifying the axes needed to support it. We do not repeat that assertion: check the coordinate system before relating azimuth to an image or map. Keep `-i 0` for our vertical example.

6.5 Impact parameter and ground

`GetImpactParameter` samples linearly between `-o` and `-O`. This reaches Fortran as `altitude`, the height of the point of closest approach to Earth's centre relative to `hground`; it is not a detector's lateral distance. `hground` is the observer altitude in the param file. Do not change all three together in the first run. The `altitude/D` branch records the passed value in metres.

SOURCE / CxRoot.cc:GetTheta, GetPhi and GetImpactParameter; conex2r.F:RunConex and conex_sub.F:Initialize2.

06 / CONFIGURATION [continued] Guide / APOEMA

Energy and sampling

Primaries and models

OBJECTIVE Use valid codes without confusing particle-numbering conventions.

Option	Default	Relevant values and examples
`-p integer`	100	Proton 100, He-4 400, C-12 1200, Fe-56 5600, photon 0 or 22, electron 11, positron -11.
`-m integer`	4	High-energy model among those actually compiled and supplied with tables.
`-H integer`	0	1 removes the electromagnetic component. Use 0 or 1; any nonzero integer becomes 1 in the parser.

Nuclei use $100 \cdot A$, with mass number A, not the PDG ion scheme. Other codes correspond to the non-nuclear particles accepted by the implementation; this does not enable every PDG particle. To change our proton example to a photon, replace `-p 100` with `-p 22`, retaining the other choices. The README excludes neutrino- and tau-initiated showers. UrQMD is the supplied configurations' low-energy model, not a valid alternative for `-m`. The parser accepts

Table 6.1: Correspondence checked in the README, source model list and code-generated help.

Code	Model	Make target	Output-name component
2	QGSJET01	qgsjet	qgsjet01
4	EPOS LHC-R	epos	eposlhcr
5	SIBYLL 2.3e	sibyll	sibyll23e
6	QGSJETII-04	qgsjetII	qgsjetII04
9	DPMJET III.2017-1	dpmjetIII	dpmjetIII
10	QGSJETIII-01	qgsjetIII	qgsjetIII

some numbers without a corresponding model; Table 6.1, library and configuration must agree.

The transition between high- and low-energy *hadronic models* differs from the *Monte Carlo/cascade-equation* threshold. The README states 80 GeV for model transitions and 30 GeV for EPOS LHC-R. Check `HiLowEgy` in the run Header. Do not use the older phrase "MC above 80 GeV" as a definition of the hybrid method.

SOURCE / README; conexHEModels.h; CxRoot.cc:GetOptions and rootOut; model paramin files.

06 / CONFIGURATION [continued] Guide / APOEMA

Energy and sampling

Seeds, saving and extra detail

OBJECTIVE Control names and reproduction without relying on silent defaults.

Syntax	Default	Meaning, example and pitfall
`-s integer`	0	Seed: choose positive, e.g. -s 314159. Zero reads /dev/urandom in this code.
`-S integer`	10	Autosave interval in events, e.g. -S 2. Must be positive; zero is used as a modulo divisor.
`-x text`	conex	Name prefix, e.g. -x proton. Neither a directory creator nor the complete filename.
`-K integer`	0	Number of successive leading-particle-chain interactions to record, e.g. -K 1. Arrays hold 100 interactions; stay well below that.
`-h`	–	Displays compiled help; does not simulate a shower.

6.6 What a seed does and does not guarantee

Autosave needs a code-level qualification: its test uses `GetEntries()+1` after `Fill`. For $S > 1$, the first call is after $S - 1$ entries, then every S entries. It does not literally guarantee a checkpoint after event S. Final writing is essential.

The README says $s \leq 0$ generates a seed automatically, with possible date and 123 fallbacks. This copy's C++ explicitly handles only $s = 0$, using `/dev/urandom`. Avoid negative values and provide a small positive integer within the parser's integer range.

The README also warns that same-seed reproducibility would be limited to QGSJet because of precision conversion. Preserve this warning even when another model matches in a local test. This edition compared SIBYLL realizations in one environment, not binary equivalence across platforms or versions. ROOT files can have different metadata despite identical event quantities.

6.7 Leading interactions

`-K 1` saves secondaries from the first interaction; `-K 2` adds those from the next interaction of the most energetic leading particle, and so on. It does not save every branch or complete genealogical generation. The `LeadingInteractions` tree is written when compiled support is enabled and $K > 0$ is requested. The runtime default is `-K 0`, without this tree. Both routes explain its branches: Python or C++. This copy's headers enable it, including momentum components and CORSIKA output codes; these are not automatically the codes accepted by `-p`.

6.8 Advanced extensions

`-X,-P,-R,-M,-T,-L,-F` depend on `CONEX_EXTENSIONS`. They are outside the tested standard build. These tools modify physics, not presentation. Read Ulrich et al. [4] before enabling them; compiled help is the reference for availability.

SOURCE / CxRoot.cc:767–773, rootOut and autosave; conexConfig.h; README comments on -s and -K.

Energy and sampling

Physics configurations and tables

OBJECTIVE Change parameters deliberately while preserving traceability.

> **CONCEPT 6.1 / Configuration file**
>
> Text read by the program to determine paths and physics parameters. Model files conex_*.param are generated from src/*.paramin by Make. Comments begin with !.

`-m 5` selects the library and `conex_sibyll.param`; its `model sibyll` and `lemodel urqmd` must agree. Energy, direction and event count are controlled by the runtime interface. Do not add instructions from another CONEX mode assuming they override the flags.

SIBYLL entry	Value	Meaning and checkable limitation
`set ixmax`	1	Gaisser–Hillas fit enabled. Zero disables the fit, not the profile.
`set hground`	0	Ground altitude in metres.
`set fehcut`	0.005	MC/CE transition fraction for hadrons.
`set feecut`	0.005	Corresponding electromagnetic fraction.
`set femcut`	0.0005	Corresponding muon fraction.
`set hacut1`	1	GeV; main hadronic/muonic profile cut.
`set hacut2/3`	1	GeV; muon/hadron profiles, each $\geq$ hacut1.
`set emcut1`	0.001	GeV; main electromagnetic cut (1 MeV).
`set emcut2/3`	0.001	GeV; photon/electron cuts, each $\geq$ emcut1.

Cuts specify which particles enter each profile; they are not MC/CE thresholds. Comments recommend at least 0.3 GeV for the hadronic cut and 1 MeV for electromagnetic cuts. Relative transition thresholds in `ConexRun` depend on energy per nucleon for nuclei and internal table limits; they do not reduce to "a fixed 80 GeV".

`xmaxp` limits simulated depth; if not set by the user, evolution proceeds to the ground for the selected geometry. The README mentions a technical maximum of $37000\,\mathrm{g\,cm^{-2}}$. Changing ground or depth can truncate the maximum. Do not change cuts or thresholds merely to accelerate the calculation without studying the observable's response.

> **EXERCISE 6.1 / Practice, without solutions**
>
> 1. Write a fixed-energy command and explain why both energy flags are needed.
> 2. Explain the difference between hacut2 and femcut without changing them.
> 3. Why is -K 2 different from saving two complete generations?

SOURCE / src/conex_sibyll.paramin; src/conex_sub.F:ConexRun; README steering files.

Energy and sampling

Check table paths

OBJECTIVE Diagnose missing tables before starting a campaign.

In param files, `fconex`, `fegs` and `furqmd` associate roles with files. Compilation substitutes `@CONEXTABROOT@` and `@BZ2@`. Paths are written into the files, usually as absolute paths; a moved package can still point to its old location.

checks before a run bash / code 6.1

```bash
cd "$CX"
printf '%s\n' "$CONEX_ROOT" "${CORSIKA_DATA:-not set}"
head -n 30 cfg/conex_sibyll.param
ls -lh tabs/CONEX/conex.wsib20.bz2
ls -lh tabs/CONEX/conex.wurq20.bz2
ls -lh tabs/UrQMD/UrQMD-1.3.1-xs.dat
ldd lib/Linux/libCONEXsibyll.so
```

6.9 What is missing and when it matters

The local inventory contains 170 files, about 79 MiB, without ready-built binaries. It includes SIBYLL 1D sources and tables, but not the large QGSJetII/QGSJetIII datasets. The SIBYLL param also references locally absent `p2dec`, `p2urq` and `p2sib` tables, read conditionally for moment/3D modes. Checking every path in every param can flag items unused by the tested 1D mode. Do not remove entries arbitrarily to silence messages.

The README points to KIT's institutional GitLab `AirShowerPhysics/corsika-data` repository [6]. For another model, obtain compatible tables, check their contents and select the root before generating param files. QGSJETIII also has a specific decompression instruction. Do not substitute an old GitHub address for the repository named by this distribution.

6.10 Compression: an investigated discrepancy

The README suggests Boost and `NO_BOOST=1`. This Makefile uses `Interface.bz2.cc`, includes `bzstream.hpp`, links `-lbz2`, and does not implement `NO_BOOST` selection. The tested build uses bzip2. If decompressing manually, filenames in param files must match; the old variable alone does not change this copy's reader.

> **NOTE / External data and redistribution**
>
> The companion package contains small original results, not CONEX or model tables. One component does not establish a licence for the whole package: the reader declares GPLv3, while UrQMD has its own notice. No unambiguous package-wide licence was found locally. Obtain the distribution through the authors' channels and preserve conditions and citations.

SOURCE / Local inventory; Fortran conditionals around p2 reads; Makefile and tabs/readLib; [6, 7].

07 / PYTHON ANALYSIS Guide / APOEMA

Read results with PyROOT

Choose one of the two routes

OBJECTIVE Open output, recognize its structure and analyse the data with Python.

We offer two ways to read the files: Python with PyROOT here, or C++ macros in section 8. Both cover the same trees, quantities and operations. **You only need one**, then continue to Everyday use. Choose this route if you prefer Python.

PyROOT is ROOT's Python interface, installed by the ROOT package in the Conda route. We use `import ROOT` and the Python standard library. In the Linux terminal, activate the environment and check access:

prepare the Python session bash / code 7.1

```
conda activate conex-apoema
python -c "import ROOT; print(ROOT.gROOT.GetVersion())"
export ARQ="$CX/first-run/first_sibyll23e_000314159_100.root"
python
```

The path in `ARQ` assumes `CX` was defined during installation. Adjust it if your simulation is elsewhere. The final command opens the Python interpreter. From then on, blocks marked Python are entered there, not in Bash.

> **CONCEPT 7.1 / Tree and branch**
>
> A TTree organizes entries (here, events or metadata). A branch is a column or group of values. An entry can contain a scalar or a vector: an entire profile belongs to one event.

Now open the file read-only and discover its contents:

open and inspect the file Python / code 7.2

```
import os
import ROOT
f = ROOT.TFile.Open(os.environ["ARQ"], "READ")
if not f or f.IsZombie():
    raise RuntimeError("Cannot open ROOT file")

f.ls()
h = f.Get("Header")
s = f.Get("Shower")
if not h or not s:
    raise RuntimeError("Header or Shower is missing")

print(h.GetEntries(), s.GetEntries())
s.Print()
h.Show(0)
```

`f.ls()` lists keys; `Get` retrieves a tree; `GetEntries` counts entries; `Print` displays the structure; `Show(0)` displays the first entry. Our first example should have one entry in each tree. Keep `f` open while using its trees.

SOURCE / ROOT Python interface [16], TFile and TTree [11, 10]; `python/conex.py`.

07 / PYTHON ANALYSIS [continued] Guide / APOEMA

Read results with PyROOT

Read types and dimensions

OBJECTIVE Choose the correct observable and avoid incorrectly sized buffers.

Table 7.1: Part of the schema implemented in output format 2.51, 1D mode.

Tree / branch	ROOT type	Meaning
Header / Seed1, Particle	Int_t	Initial seed and primary code.
Header / lgEmin, lgEmax	Double_t	Bounds of $\log_{10}(E/\mathrm{eV})$.
Header / Version	Float_t	Recorded program identifier, distinct from OutputVersion.
Header / OutputVersion	Float_t	Independent output-format version.
Header / HEModel, LEModel	Int_t	Effective interaction-model codes.
Header / HiLowEgy	Float_t	Hadronic-model transition energy, GeV.
Shower / lgE, zenith, azimuth	Float_t	Logarithmic energy and angles in degrees.
Shower / altitude	Double_t	Impact parameter in metres.
Shower / nX	Int_t	Number of points in each profile.
Shower / X, H, D	Float_t[nX]	Depth, altitude and distance along the axis.
Shower / N, Mu, Electrons	Float_t[nX]	Populations above the relevant cuts.
Shower / Gamma, Hadrons	Float_t[nX]	Photons and hadrons in their own profiles.
Shower / dEdX, dMu	Float_t[nX]	Energy deposit per depth and muon production.
Shower / EGround	Float_t[3]	Energy in GeV: e+γ, hadrons, muons at the final boundary.

Float_t and Double_t are single- and double-precision numbers; Int_t is integer. Leaflist /F, /D and /I identify these types. Do not bind a double address to an /F branch. `TTreeReaderArray<Float_t>` handles variable-length arrays without assuming a 5000- or 20000-point buffer.

7.1 Other fields requiring care

`Xfirst` is first-interaction depth in $\mathrm{g\,cm^{-2}}$; `Hfirst` is its altitude in metres. Despite its name, `XfirstIn` is inelasticity, not depth. Seed2 and Seed3 store generator-state counters, not two new user-selected seeds. cpuTime is event CPU time in seconds, not necessarily the entire initialization cost. SVNRevision, still listed in the README, is not created by the current C++ tree writer.

SOURCE / CxRoot.cc:923–944 and 1042–1077; README; schema recorded during inspection.

7.2 Check the leaf before copying its vector

Still in Python, with `f` and `s` open, run:

profile type and dimension Python / code 7.3

```python
folha = s.GetBranch("X").GetListOfLeaves().At(0)
print(folha.GetTypeName())
print(folha.GetLeafCount().GetName())
s.GetEntry(0)
print(int(s.nX), float(s.lgE), float(s.zenith))
```

For `X`, the studied output reports `Float_t` and counter `nX`. `GetEntry(0)` loads the first event. Vectors are updated when another entry is loaded; copy their values if you need to keep them.

07 / PYTHON ANALYSIS [continued]
Guide / APOEMA

Read results with PyROOT

Plot a profile with Python

OBJECTIVE Copy an event's vectors and plot quantities with the correct units.

Separate depth and charged-particle count for the first event. In the Python session where the file remains open, run:

copy one event — Python / code 7.4

```
s.GetEntry(0)
n = int(s.nX)
x = [float(s.X[i]) for i in range(n)]
ncar = [float(s.N[i]) for i in range(n)]
dep = [float(s.dEdX[i]) for i in range(n - 1)]
meio = [(x[i] + x[i+1])/2 for i in range(n - 1)]
integral = sum(dep[i]*(x[i+1]-x[i]) for i in range(n-1))
print("Deposit over intervals [GeV]:", integral)
```

There are `nX` planes for `X` and `N`, but only `nX-1` intervals between them. `dEdX[i]` represents the interval from `X[i]` to `X[i+1]`; use its midpoint without extrapolating the final point. The sum integrates over recorded intervals, not a detector measurement.

A `TGraph` plots ordered pairs; `TCanvas` is the drawing surface. Continue to save the charged-particle profile:

plot and export without extra libraries — Python / code 7.5

```
from array import array
ROOT.gROOT.SetBatch(True)
c = ROOT.TCanvas("c", "Profile", 800, 500)
g = ROOT.TGraph(n, array("d", x), array("d", ncar))
g.SetTitle("Charged particles;X [g/cm^{2}];N(X)")
g.Draw("AL")
c.Print("charged-profile.pdf")
f.Close()
exit()
```

This writes a PDF in the current directory; `exit()` returns to the terminal. To obtain both plots, the CSV and integral with checks of types, dimensions and finite values, use the supplied script. Open the terminal at the downloaded repository root and run:

complete PyROOT profile — bash / code 7.6

```
python python/conex.py profile "$ARQ" --output profile-study
```

On GitHub: `python/conex.py`.

It creates `profile-study.pdf` and `profile-study.csv`; this prefix preserves section 5's output. Choose a different prefix to repeat an analysis: overwriting is refused. Compare units and curve shapes with the first profile.

SOURCE / `python/conex.py`: read_profile and profile; README dEdX positioning; ROOT TGraph [17].

Read results with PyROOT

Histograms and summaries with Python

OBJECTIVE Distinguish a single-event profile from a distribution of maxima across events.

> CONCEPT 7.2 / Histogram
>
> Groups values into intervals called bins. The height can represent counts, fractions or densities: state which. Normal ROOT bins start at 1; bin 0 is underflow and the extra final bin is overflow.

`XmxdEdX` estimates each event's deposit maximum by quadratic interpolation. Its histogram has **one value per shower**; do not fill it with every element of `X`.

Generate an educational sample of 12 protons and 12 iron nuclei, at 10^{17} eV, vertical, with distinct seeds and the same models and configurations. At the repository root with the environment active and `CX` defined, run:

prepare a small sample — bash / code 7.7

```bash
bash scripts/campaign.sh "$CX" "$PWD/my-campaign"
export A="$PWD/my-campaign/proton_sibyll23e_000271828_100.root"
export B="$PWD/my-campaign/iron_sibyll23e_000271829_5600.root"
python
```

The destination must be new. The `script` records parameters, configuration and logs. If the campaign exists, skip the first line and set `A` and `B` to your existing files. In Python, plot the proton distribution:

one value per event — Python / code 7.8

```python
import os, ROOT
ROOT.gROOT.SetBatch(True)
f = ROOT.TFile.Open(os.environ["A"], "READ")
s = f.Get("Shower")
hX = ROOT.TH1D("hX", ";Xmax dE/dX [g/cm^{2}];Eventos", 30, 0, 1200)
s.Draw("XmxdEdX>>hX", "", "goff")
hX.SetDirectory(0)
print(hX.GetEntries(), hX.GetBinContent(0), hX.GetBinContent(31))
c = ROOT.TCanvas("cHist", "Maxima", 800, 500)
hX.Draw("HIST")
c.Print("interactive-histogram.pdf")
f.Close()
exit()
```

The 30 bins are $40\,\mathrm{g\,cm^{-2}}$ wide. Check out-of-range counts: bin 0 is underflow and 31 is overflow. For a summary selecting finite positive values, run in the terminal:

histogram and sample summary — bash / code 7.9

```bash
python python/conex.py histogram "$A" --output histogram-python
```

The PDF shows counts. The CSV contains accepted and rejected entries, mean, standard deviation, standard error and external bins. The summary needs two valid events and includes accepted maxima outside the plotted range.

SOURCE / `python/conex.py`: maxima, summary and histogram; ROOT TH1 [12]; `scripts/campaign.sh`.

07 / PYTHON ANALYSIS [continued] Guide / APOEMA

Read results with PyROOT

Compare samples with Python

OBJECTIVE Compare samples while holding conditions outside the question fixed.

Compare the deposit maxima in the previous campaign's two samples. Only the primary should differ: check total energy, direction, atmosphere, cuts and models in the configurations and `Header`. At the repository root, with `A` and `B` pointing to the files, run:

compare with PyROOT bash / code 7.10

```bash
python python/conex.py compare "$A" "$B" --output comparison-python
```

On GitHub: `python/conex.py`.

The script creates `comparison-python.pdf` and `comparison-python.csv`. Both groups use the same bin edges. The first file is labelled `proton` and the second `iron`: preserve that order.

7.3 What the numbers summarize

For accepted values x_i, calculate directly:

$$\bar{x} = \frac{1}{n}\sum_i x_i, \qquad s = \sqrt{\frac{\sum_i (x_i - \bar{x})^2}{n-1}}, \qquad u(\bar{x}) \simeq \frac{s}{\sqrt{n}}.$$

Sample standard deviation s describes event-to-event spread. $u(\bar{x})$ estimates the mean's statistical uncertainty for independent finite-variance events. They are different quantities, and neither summarizes model or atmospheric systematic effects.

One realization of this small campaign gave:

Group	n	Mean	s	Interpretation
Proton	12	729.28	108.56	Greater spread in this small sample.
Iron	12	580.93	29.12	Smaller average depth of maximum in this sample.

Mean and standard deviation are in $\mathrm{g\,cm^{-2}}$. Fluctuations and limited statistics make these values realization dependent. This exercises the workflow, not a precision conclusion about cosmic-ray composition. No Gaussian fit was used to obtain the means.

For unequal sample sizes, raw counts do not compare probabilities. Use a common normalization and check external bins before showing fractions or densities. More events require more time and storage; estimate resources with a small sample first.

> **EXERCISE 7.1 / Practice, without solutions**
>
> 1. Compare standard deviation and standard error in the CSV and explain their roles.
> 2. Change the bins in the script's hist function. Does the summary from individual values change? Use a new output prefix.
> 3. Plan a comparison between energies and state which parameters should remain fixed.

SOURCE / `python/conex.py`: compare; `results/comparison-summary.csv` and provenance.

07 / PYTHON ANALYSIS [continued] Guide / APOEMA

Read results with PyROOT

Which maximum was calculated?

OBJECTIVE Distinguish estimators and interpret fit parameters with their limitations.

Branch	Interpretation in this output
`Xmax, Nmax`	Maximum and amplitude of the extended Gaisser–Hillas fit to charged-particle count.
`Xmx, Nmx`	Maximum and amplitude from local quadratic interpolation of $N(X)$.
`XmxdEdX, dEdXmx`	Maximum and amplitude from local quadratic interpolation of energy deposit.
`X0, p1, p2, p3`	Parameters of the extended Gaisser–Hillas form. X0 is not Xfirst.

The README describes CONEX's supplied fit function as

$$N(X) = N_{\max}\left(\frac{X - X_0}{X_{\max} - X_0}\right)^{(X_{\max}-X_0)/\lambda(X)} \exp\left[\frac{X_{\max} - X}{\lambda(X)}\right], \qquad \lambda(X) = p_1 + p_2 X + p_3 X^2. \tag{7.1}$$

$N_{\max}$ is a count; X, X_0, $X_{\max}$ and p_1 are in $\mathrm{g\,cm^{-2}}$; p_2 is dimensionless and p_3 has inverse-depth units. The usual interpretation requires $X > X_0$, $X_{\max} > X_0$ and $\lambda(X) > 0$. Do not evaluate it blindly outside that domain.

7.4 Minimum checks before comparing fits

Confirm `ixmax=1`; check finite values, a maximum inside the simulated range, and profile points on both sides. Compare curve and data visually, especially for double structures or ground truncation. The described `chi2` is $\chi^2/\mathrm{ndf}/\sqrt{N_{\max}}$, not a conventional reduced chi-square. A small value alone does not establish an excellent fit or justify a p-value without an error model.

Our histograms use `XmxdEdX`, without fitting a new function to the histogram. Label the estimator on axes and in text: "deposit maximum from quadratic interpolation", rather than calling every field simply Xmax.

> **EXAMPLE 7.1 / Why two maxima need not coincide**
>
> The $N(X)$ maximum counts particles above cuts; the deposit maximum weights energy transferred to the medium. Changing cuts can shift one without the same shift in the other. A difference is not by itself a bug: first identify the branches and profiles being compared.

SOURCE / README steering files and Shower description; CxRoot.cc:interpolateProfile; plotProfile.C:GaisserHillas.

07 / PYTHON ANALYSIS [continued]
Guide / APOEMA

Read results with PyROOT

Leading interactions: what to save

OBJECTIVE Create the optional tree and distinguish its two counters.

The first file was generated without `-K` and has no LeadingInteractions tree. This tree is written only if compiled support is enabled and runtime `-K` is greater than zero. Support is already enabled in the studied distribution; the runtime default is `-K 0`.

Save up to two successive interactions of the leading chain. After each collision, follow the most energetic leading particle, not all branches or complete generations. Run this once. If the file exists, reuse it and skip the block; if only the directory exists, choose a different name before proceeding.

In the Linux terminal with the environment and CX prepared, run:

one event with leading-chain detail — bash / code 7.11

```bash
cd "$CX"
mkdir leading-run
export CONEX_ROOT="$CX/cfg"
export ROOT_OUT="$CX/leading-run"
bin/conex2r -m 5 -p 100 -e 17 -E 17 -z 0 -Z 0 \
  -n 1 -s 424242 -K 2 -x leading > leading-run/run.log 2>&1
```

The file is `leading_sibyll23e_000424242_100.root`. Its link leads to the recipe in `results`; generate it locally. Check the log and open the output before analysis. One tree entry corresponds to an event: `nInt` counts interactions actually recorded, while `nPart` counts particles in the complete stored list. The requested `-K` does not guarantee a chain of that length.

7.5 Per-interaction branches

Branch	Type/dimension	Meaning and unit
`nInt`	`Int_t`	Number of stored interactions in the event.
`mult`	`Int_t[nInt]`	Secondary multiplicity per interaction; count.
`pId`	`Int_t[nInt]`	Species code of the particle initiating each interaction.
`pEnergy`	`Double_t[nInt]`	Its energy in GeV.
`matg`	`Int_t[nInt]`	Target-nucleus mass number.
`depth`	`Double_t[nInt]`	Interaction depth, $\mathrm{g\,cm^{-2}}$.
`height`	`Double_t[nInt]`	Interaction altitude in metres.
`kinel`	`Double_t[nInt]`	Dimensionless inelasticity field; see the qualification below.

When active, kinel uses $1 - E_{\mathrm{largest}} / \sum E_{\mathrm{secondaries}}$. In this implementation, its calculation depends on first-interaction initialization. Zeros in later interactions do not imply perfectly elastic collisions. Start with the first interaction when studying this quantity.

SOURCE / CxRoot.cc: LeadingInteractions; leadingInteractionsData.cc; conex_sub.F: cxafinal and lxfirstIn control.

07 / PYTHON ANALYSIS [continued]
Guide / APOEMA

Read results with PyROOT

Secondaries and their interaction of origin

OBJECTIVE Associate particles with interactions without confusing vector indices.

Branch	Type/dimension	Meaning and unit
`nPart`	`Int_t`	Total particles in the event's stored list.
`Energy`	`Double_t[nPart]`	Each secondary's energy in GeV.
`Type`	`Int_t[nPart]`	Each secondary's species code.
`idInt`	`Int_t[nPart]`	Originating interaction number, starting at 1 in this copy.
`px, py, pz`	`Double_t[nPart]`	Momentum components in GeV/c, when enabled at compilation.

Index 0 in the multiplicity vector is the first interaction. In contrast, `idInt[j] == 1` selects its secondaries and `idInt[j] == 2` selects the second interaction's particles. Never use a particle index directly to address an nInt-length vector.

Particles are grouped by interaction and sorted by energy in this copy's output. Use idInt rather than position to associate them. mult[i] describes one interaction; nPart describes the complete list. Without truncation, counting idInt values by group checks multiplicity. Reassess that equality if the log warns of a storage limit.

> **NOTE / Output codes differ from input codes**
>
> LEADING_INTERACTIONS_CORSIKA is enabled in this copy: pId and Type use CORSIKA numbering. Do not automatically interpret them as PDG codes or as the nuclear input codes of -p. Check the convention before identifying a species from its number.

Spelling matters: branch `mult` contains leaf `Mult`; `matg` contains `Matg`. A leaf describes the stored value's type and dimensions. Retrieve it through the branch's leaf list when names differ.

07 / PYTHON ANALYSIS [continued]
Guide / APOEMA

Read results with PyROOT

Read multiplicities with Python

Return to the repository root and set the new file in the terminal:

file with the leading chain — bash / code 7.12

```bash
export LIDER="$CX/leading-run/leading_sibyll23e_000424242_100.root"
python
```

On GitHub: leading-interaction recipe.

In Python, open the tree and count secondaries by interaction:

multiplicity and interaction association — Python / code 7.13

```python
import os, ROOT
from collections import Counter
f = ROOT.TFile.Open(os.environ["LIDER"], "READ")
t = f.Get("LeadingInteractions")
if not t:
    raise RuntimeError("Missing tree: check -K and the build")

t.Print()
t.GetEntry(0)
origens = Counter(int(t.idInt[j]) for j in range(int(t.nPart)))
for i in range(int(t.nInt)):
    print(i+1, int(t.mult[i]), origens[i+1])

f.Close()
exit()
```

Each line shows the interaction, multiplicity and count of associated particles. The reference example has 115 secondaries in the first interaction and 571 in the second: `nInt` = 2, `nPart` = 686. Vector indices start at 0; `idInt` values start at 1.

To repeat the reading with checks and save a per-interaction summary, run:

interaction summary with PyROOT — bash / code 7.14

```bash
python python/conex.py interactions "$LIDER" --output interactions-python
```

On GitHub: python/conex.py.

The CSV records multiplicity, stored-particle count, projectile energy, inelasticity, target, depth and altitude. The script obtains the leaf through its branch, avoiding confusion between `mult` and `Mult`. Continue to Everyday use; the next section repeats the operations in C++.

SOURCE / python/conex.py: read_interactions and interactions; CONEX sources identified in the tables.

08 / C++ ANALYSIS
Guide / APOEMA

Read results with C++

An alternative to Python

OBJECTIVE Discover the actual file structure instead of assuming names from an earlier manual.

> **CONCEPT 8.1 / Tree and branch**
>
> A TTree organizes entries (events or metadata). A branch is a column or group of values. An entry can contain a scalar or a vector: an entire profile belongs to one event.

This is the second reading route. If you completed section 7, skip this section and continue to Everyday use. Both cover structure, profiles, histograms, comparisons and leading interactions; there is no need to execute both.

Open the Linux terminal at the downloaded repository root, with ROOT available and `CX` defined. Use the first shower's file:

open output in the C++ interpreter — bash / code 8.1

```bash
export ARQ="$CX/first-run/first_sibyll23e_000314159_100.root"
root -l "$ARQ"
```

On GitHub: first-event recipe.

The command opens ROOT and makes the file available as `_file0`. Enter this C++ block in ROOT:

interactive exploration — C++ / code 8.2

```cpp
_file0->ls();
auto h = _file0->Get<TTree>("Header");
auto s = _file0->Get<TTree>("Shower");
h->Print();
s->Print();
s->GetEntries();
h->Show(0);
s->Show(0);
s->GetLeaf("X")->GetTypeName();
s->GetLeaf("X")->GetLeafCount()->GetName();
.q
```

These calls assume the trees and leaves exist, after inspecting `ls`/`Print`. Reusable macros should check pointers first; `inspect.C` does so for trees. Here `X` is `Float_t` with counter `nX`; inspect other distributions afresh.

8.1 Cycles are not independent backup trees

`Header;1` and `Header;2` are ROOT persistence-key cycles for the same object name. Autosave and final writing can create cycles. There is no rule of "four standard trees, two backups". `Get("Shower")` returns the latest cycle; `Get("Shower;1")` selects one explicitly. Do not add cycle entries as though they were new events.

SOURCE / CxRoot.cc:rootOut; official TTree and TFile documentation [10, 11]; `macros/inspect.C`.

08 / C++ ANALYSIS [continued]
Guide / APOEMA

Read results with C++

Read types and dimensions

OBJECTIVE Choose the correct observable and avoid incorrectly sized buffers.

Table 8.1: Part of the schema implemented in output format 2.51, 1D mode.

Tree / branch	ROOT type	Meaning
Header / Seed1, Particle	Int_t	Initial seed and primary code.
Header / lgEmin, lgEmax	Double_t	Bounds of $\log_{10}(E/\mathrm{eV})$.
Header / Version	Float_t	Recorded program identifier, distinct from OutputVersion.
Header / OutputVersion	Float_t	Independent output-format version.
Header / HEModel, LEModel	Int_t	Effective interaction-model codes.
Header / HiLowEgy	Float_t	Hadronic-model transition energy, GeV.
Shower / lgE, zenith, azimuth	Float_t	Logarithmic energy and angles in degrees.
Shower / altitude	Double_t	Impact parameter in metres.
Shower / nX	Int_t	Number of points in each profile.
Shower / X, H, D	Float_t[nX]	Depth, altitude and distance along the axis.
Shower / N, Mu, Electrons	Float_t[nX]	Populations above the relevant cuts.
Shower / Gamma, Hadrons	Float_t[nX]	Photons and hadrons in their own profiles.
Shower / dEdX, dMu	Float_t[nX]	Energy deposit per depth and muon production.
Shower / EGround	Float_t[3]	Energy in GeV: e+γ, hadrons, muons at the final boundary.

Float_t and Double_t are single- and double-precision numbers; Int_t is integer. Leaflist /F, /D and /I identify these types. Do not bind a double address to an /F branch. `TTreeReaderArray<Float_t>` handles variable-length arrays without assuming a 5000- or 20000-point buffer.

8.2 Other fields requiring care

`Xfirst` is first-interaction depth in $\mathrm{g\,cm^{-2}}$; `Hfirst` is its altitude in metres. Despite its name, `XfirstIn` is inelasticity, not depth. Seed2 and Seed3 store generator-state counters, not two new user-selected seeds. cpuTime is event CPU time in seconds, not necessarily the entire initialization cost. SVNRevision, still listed in the README, is not created by the current C++ tree writer.

SOURCE / CxRoot.cc:923–944 and 1042–1077; README; schema recorded during inspection.

> **NOTE / Learn from the original macros**
>
> The distributed plotProfile.C illustrates branch association and deposit positions. Its TH1 filling uses index zero (underflow), however; other macros use old syntax or look for FirstInteraction. Treat them as historical references and check each operation, not as automatic compatibility tests.

08 / C++ ANALYSIS [continued] Guide / APOEMA

Read results with C++

Plot a single-event profile

OBJECTIVE Use correct units and distinguish planes from intervals.

profile.C opens Shower, checks the entry, reads nX, X, N and dEdX with explicit types and checks their lengths. Event zero is the first entry. TGraph represents ordered pairs; TCanvas is the drawing and export surface.

The following is an illustrative excerpt: **do not execute it on its own**. The complete macro defines t and selects the entry; run it with the command below.

illustrative excerpt - do not execute alone C++ / code 8.3

```
TTreeReader r(t);
TTreeReaderValue<Int_t> n(r,"nX");
TTreeReaderArray<Float_t> x(r,"X"), ncar(r,"N"), dep(r,"dEdX");
// After selecting the entry and checking the reader:
for (int i=0; i+1<*n; ++i) {
  double atual = x[i], proximo = x[i+1];
  double meio = (atual+proximo)/2.;
  double largura = proximo-atual;
  // N[i] is at plane X[i]; dep[i] refers to the next interval.
}
```

8.3 The deposit has a different position

The README and authors' macro place dEdX[i] halfway between X[i] and X[i+1]. There are nX planes and $nX-1$ intervals. Our macro uses only those intervals and their individual midpoints, without extrapolating the final point or assuming uniform spacing.

It calculates $E_{\mathrm{dep,intervals}} \simeq \sum_{i=0}^{nX-2} (dE/dX)_i (X_{i+1} - X_i)$ in GeV. This need not equal the initial energy: energy at the final boundary, invisible energy and loss/boundary conventions matter. It is not a detector's calorimetric calibration.

8.4 Run the complete macro

After leaving the interpreter with .q, run in the terminal at the repository root:

write profile and CSV with C++ bash / code 8.4

```
root -l -b -q "macros/profile.C(\"$ARQ\",0,\"profile-cpp\")"
```

Argument zero selects the first entry. Prefix profile-cpp determines the PDF and CSV names; use a new prefix for each analysis. -b avoids windows and -q quits afterwards.

8.5 What the macro produces

profile-cpp.pdf contains $N(X)$ and deposit plots; profile-cpp.csv contains the values, with no deposit interval associated with the final plane. Existing outputs are refused. Per-event CSVs are not redistributed with the package because of the UrQMD event-sharing conditions.

SOURCE / macros/profile.C; original plotProfile.C; README profiles.

08 / C++ ANALYSIS [continued]
Guide / APOEMA

Read results with C++

Histograms and simple summaries

OBJECTIVE Build a distribution without confusing physical spread and uncertainty of the mean.

> CONCEPT 8.2 / Histogram
>
> Groups values into bins. The height can be counts, fractions or densities: say which. ROOT's normal bins start at 1; bin 0 is underflow and the extra final bin is overflow.

A histogram of `XmxdEdX` collects one deposit maximum per event, not one event's longitudinal profile. Use 12 protons and 12 iron nuclei at 10^{17} eV, vertical, with identical settings and distinct seeds. At the repository root, run:

small sample and ROOT session — bash / code 8.5

```bash
bash scripts/campaign.sh "$CX" "$PWD/my-campaign"
export A="$PWD/my-campaign/proton_sibyll23e_000271828_100.root"
export B="$PWD/my-campaign/iron_sibyll23e_000271829_5600.root"
root -l "$A"
```

On GitHub: `scripts/campaign.sh`; campaign recipe.

The destination must be new. If the campaign exists, skip the first line and adjust `A` and `B`. In ROOT, retrieve the tree with `auto s = _file0->Get<TTree>("Shower");`, check `s->Print();`, then enter:

inspect a histogram — C++ / code 8.6

```cpp
auto hX = new TH1D("hX",";Xmax dE/dX [g/cm^{2}];Eventos",30,0,1200);
s->Draw("XmxdEdX>>hX", "", "goff");
hX->Draw("HIST");
hX->GetEntries();
hX->GetBinContent(0);       // Underflow
hX->GetBinContent(31);      // Overflow: 30 normal bins
```

This is direct exploration. `compare.C` adds checks and excludes non-finite or non-positive values, counting them separately. It does not fit a Gaussian to estimate a mean: summaries come directly from all accepted event values, including those outside the plotted range. Leave with `.q` and generate the PDF and summary CSV in the terminal:

histogram and summary with C++ — bash / code 8.7

```bash
root -l -b -q "macros/histogram.C(\"$A\",\"histogram-cpp\")"
```

On GitHub: `macros/histogram.C`; `compare.C (dependency)`.

The macro uses 30 bins from 0 to $1200\,\mathrm{g\,cm^{-2}}$ and requires at least two valid events for sample standard deviation. Summaries include accepted underflow and overflow values.

SOURCE / `macros/histogram.C` and `compare.C`; TH1 and TTree [12, 10].

08 / C++ ANALYSIS [continued] Guide / APOEMA

Read results with C++

Compare samples with C++

OBJECTIVE Compare distributions using the same units, selection and bins.

Use the campaign's proton and iron files. Check total energy, direction, atmosphere, cuts and models in the configurations and Header; only the primary should differ. At the repository root, run:

compare and save summaries bash / code 8.8

```bash
root -l -b -q "macros/compare.C(\"$A\",\"$B\",\"comparison-cpp\")"
```

On GitHub: macros/compare.C.

The first input is labelled proton and the second iron. Outputs are `comparison-cpp.pdf` and `comparison-cpp.csv`; choose a new prefix when repeating, because overwriting is refused.

8.6 Mean, spread and standard error

$$\bar{x} = \frac{1}{n}\sum_i x_i, \qquad s = \sqrt{\frac{\sum_i (x_i - \bar{x})^2}{n-1}}, \qquad u(\bar{x}) \simeq \frac{s}{\sqrt{n}}. \tag{8.1}$$

The sample standard deviation describes event-to-event spread. The standard error estimates the mean's statistical uncertainty for independent finite-variance events. It is not a systematic uncertainty, nor a stable estimate of tails from a dozen showers.

8.7 Keep the physical question intact

Use identical energy, angle, cuts, atmosphere and models, changing only the investigated factor. Keep bins and limits equal. For unequal sample sizes, raw counts do not compare probabilities: normalize by the integral and state whether the result is a fraction or density per depth. Check underflow/overflow first.

One realization with 12 events per primary gave a mean of $729.28\,\mathrm{g\,cm^{-2}}$ and sample standard deviation $108.56\,\mathrm{g\,cm^{-2}}$ for protons; iron gave 580.93 and $29.12\,\mathrm{g\,cm^{-2}}$. In this small sample, iron's average maximum is shallower and its spread is smaller.

These numbers demonstrate the workflow and depend on the realization. A dozen showers cannot support precision composition conclusions or small model differences. Means are calculated from event values, not Gaussian histogram fits.

> **EXERCISE 8.1 / Practice, without solutions**
>
> 1. Use Print to find the Mu array's type and counter in your file.
> 2. Change the number of bins. What should stay unchanged in the summary computed directly from events?
> 3. Why does averaging all values of X answer a different question from averaging XmxdEdX?

SOURCE / macros/compare.C; results/comparison-summary.csv and provenance.

08 / C++ ANALYSIS [continued] Guide / APOEMA

Read results with C++

Which maximum was calculated?

OBJECTIVE Distinguish estimators and interpret fit parameters with their limitations.

Branch	Interpretation in this output
`Xmax, Nmax`	Maximum and amplitude of the extended Gaisser–Hillas fit to charged-particle count.
`Xmx, Nmx`	Maximum and amplitude from local quadratic interpolation of $N(X)$.
`XmxdEdX, dEdXmx`	Maximum and amplitude from local quadratic interpolation of energy deposit.
`X0, p1, p2, p3`	Parameters of the extended Gaisser–Hillas form. X0 is not Xfirst.

The README describes CONEX's supplied fit function as

$$N(X) = N_{\max}\left(\frac{X - X_0}{X_{\max} - X_0}\right)^{(X_{\max}-X_0)/\lambda(X)} \exp\left[\frac{X_{\max} - X}{\lambda(X)}\right], \qquad \lambda(X) = p_1 + p_2 X + p_3 X^2. \tag{8.2}$$

$N_{\max}$ is a count; X, X_0, $X_{\max}$ and p_1 are in $\mathrm{g\,cm^{-2}}$; p_2 is dimensionless and p_3 has inverse-depth units. The usual interpretation requires $X > X_0$, $X_{\max} > X_0$ and $\lambda(X) > 0$. Do not evaluate it blindly outside that domain.

8.8 Minimum checks before comparing fits

Confirm `ixmax=1`; check finite values, a maximum inside the simulated range, and profile points on both sides. Compare curve and data visually, especially for double structures or ground truncation. The described `chi2` is $\chi^2/\mathrm{ndf}/\sqrt{N_{\max}}$, not a conventional reduced chi-square. A small value alone does not establish an excellent fit or justify a p-value without an error model.

Our histograms use `XmxdEdX`, without fitting a new function to the histogram. Label the estimator on axes and in text: "deposit maximum from quadratic interpolation", rather than calling every field simply Xmax.

> **EXAMPLE 8.1 / Why two maxima need not coincide**
>
> The $N(X)$ maximum counts particles above cuts; the deposit maximum weights energy transferred to the medium. Changing cuts can shift one without the same shift in the other. A difference is not by itself a bug: first identify the branches and profiles being compared.

SOURCE / README steering files and Shower description; CxRoot.cc:interpolateProfile; plotProfile.C:GaisserHillas.

08 / C++ ANALYSIS [continued] Guide / APOEMA

Read results with C++

Leading interactions: what to save

OBJECTIVE Create the optional tree and distinguish its two counters.

The first file was generated without -K and has no LeadingInteractions tree. This tree is written only if compiled support is enabled and runtime -K is greater than zero. Support is already enabled in the studied distribution; the runtime default is -K 0.

Save up to two successive interactions of the leading chain. After each collision, follow the most energetic leading particle, not all branches or complete generations. Run this once. If the file exists, reuse it and skip the block; if only the directory exists, choose a different name before proceeding.

In the Linux terminal with the environment and CX prepared, run:

one event with leading-chain detail bash / code 8.9

```bash
cd "$CX"
mkdir leading-run
export CONEX_ROOT="$CX/cfg"
export ROOT_OUT="$CX/leading-run"
bin/conex2r -m 5 -p 100 -e 17 -E 17 -z 0 -Z 0 \
  -n 1 -s 424242 -K 2 -x leading > leading-run/run.log 2>&1
```

The file is leading_sibyll23e_000424242_100.root. Its link leads to the recipe in results; generate it locally. Check the log and open the output before analysis. One tree entry corresponds to an event: nInt counts interactions actually recorded, while nPart counts particles in the complete stored list. The requested -K does not guarantee a chain of that length.

8.9 Per-interaction branches

Branch	Type/dimension	Meaning and unit
nInt	Int_t	Number of stored interactions in the event.
mult	Int_t[nInt]	Secondary multiplicity per interaction; count.
pId	Int_t[nInt]	Species code of the particle initiating each interaction.
pEnergy	Double_t[nInt]	Its energy in GeV.
matg	Int_t[nInt]	Target-nucleus mass number.
depth	Double_t[nInt]	Interaction depth, $\mathrm{g\,cm^{-2}}$.
height	Double_t[nInt]	Interaction altitude in metres.
kinel	Double_t[nInt]	Dimensionless inelasticity field; see the qualification below.

When active, kinel uses $1 - E_{\mathrm{largest}} / \sum E_{\mathrm{secondaries}}$. In this implementation, its calculation depends on first-interaction initialization. Zeros in later interactions do not imply perfectly elastic collisions. Start with the first interaction when studying this quantity.

SOURCE / CxRoot.cc: LeadingInteractions; leadingInteractionsData.cc; conex_sub.F: cxafinal and lxfirstIn control.

08 / C++ ANALYSIS [continued]
Guide / APOEMA

Read results with C++

Secondaries and their interaction of origin

OBJECTIVE Associate particles with interactions without confusing vector indices.

Branch	Type/dimension	Meaning and unit
`nPart`	`Int_t`	Total particles in the event's stored list.
`Energy`	`Double_t[nPart]`	Each secondary's energy in GeV.
`Type`	`Int_t[nPart]`	Each secondary's species code.
`idInt`	`Int_t[nPart]`	Originating interaction number, starting at 1 in this copy.
`px, py, pz`	`Double_t[nPart]`	Momentum components in GeV/c, when enabled at compilation.

Index 0 in the multiplicity vector is the first interaction. In contrast, `idInt[j] == 1` selects its secondaries and `idInt[j] == 2` selects the second interaction's particles. Never use a particle index directly to address an nInt-length vector.

Particles are grouped by interaction and sorted by energy in this copy's output. Use idInt rather than position to associate them. mult[i] describes one interaction; nPart describes the complete list. Without truncation, counting idInt values by group checks multiplicity. Reassess that equality if the log warns of a storage limit.

> **NOTE / Output codes differ from input codes**
>
> LEADING_INTERACTIONS_CORSIKA is enabled in this copy: pId and Type use CORSIKA numbering. Do not automatically interpret them as PDG codes or as the nuclear input codes of -p. Check the convention before identifying a species from its number.

Spelling matters: branch `mult` contains leaf `Mult`; `matg` contains `Matg`. A leaf describes the stored value's type and dimensions. Retrieve it through the branch's leaf list when names differ.

08 / C++ ANALYSIS [continued]
Guide / APOEMA

Read results with C++

Read multiplicities with C++

At the repository root, set the path and open the file in the terminal:

open the detailed event — bash / code 8.10

```bash
export LIDER="$CX/leading-run/leading_sibyll23e_000424242_100.root"
root -l "$LIDER"
```

On GitHub: leading-interaction recipe.

Inside ROOT, enter:

obtain the leaf through its branch — C++ / code 8.11

```cpp
auto t = _file0->Get<TTree>("LeadingInteractions");
if (!t) throw std::runtime_error("Missing tree: check -K");
t->Print();
t->GetEntry(0);
auto b = t->GetBranch("mult");
auto m = static_cast<TLeaf*>(b->GetListOfLeaves()->At(0));
std::cout << m->GetName() << " " << m->GetTypeName() << std::endl;
for (int i=0; i<m->GetLen(); ++i) {
  std::cout << "interaction " << i+1 << ": " << m->GetValue(i) << std::endl;
}
.q
```

The leaf name is Mult; its vector has nInt elements. The reference event has multiplicities 115 and 571, nInt=2 and nPart=686. To check counts grouped by idInt and save the summary, run in the terminal:

interaction summary with C++ — bash / code 8.12

```bash
root -l -b -q "macros/interactions.C(\"$LIDER\",0,\"interactions-cpp\")"
```

On GitHub: macros/interactions.C.

The macro checks types, both lengths and the association idInt=i+1. The CSV contains projectile energy, multiplicity, stored-particle count, inelasticity, target, depth and altitude for each interaction. Continue to Everyday use; you do not need the Python route as well.

SOURCE / macros/interactions.C; CONEX sources identified in the tables.

09 / ROUTINE Guide / APOEMA

Everyday use

Record the context as part of the simulation

OBJECTIVE Repeat an analysis without relying on the operator's memory.

Use a new directory per campaign, with a manifest recording model, primary, energy, angles, event count, seed and configuration. Filenames help, but do not replace the manifest: energy and direction are absent from this interface's automatic filename.

suggested organization - not commands / code 9.1

```
campaign-energy17/
  environment.txt       # System, tools, relevant variables
  campaign.csv          # Parameters for each run
  used-config.param     # Exact configuration read
  proton.log, iron.log  # One log per run
  *.root                # Local results, not redistributed here
  SHA256SUMS            # Input/output integrity
analysis-energy17/
  comparison.pdf, comparison.csv
  command.txt           # Exact analysis invocation
```

9.1 Seeds and independence

Choose distinct positive seeds and record them. Restarting every event with the same seed can repeat it; a run with -n 12 advances the generator. For independent jobs, record the job–seed–command correspondence. Do not rely only on the clock to prevent collisions among simultaneous runs.

9.2 Useful logs

> run.log 2>&1 saves normal output and errors. Read the start (configuration and initialization) and end (completion and errors). Scripts use set -euo pipefail: command failures stop execution. With tee, use pipefail to preserve a simulation failure through the pipeline; tee succeeding alone does not validate the simulator.

9.3 Combine files without losing metadata

TChain can read multiple trees of the same schema. Check Header before mixing runs: one Header entry per file does not align with one Shower entry per event. The original merge.C still uses FirstInteraction and was not adopted for the current tree.

EXERCISE 9.1 / Practice, without solutions

1. Create a manifest for three energies and two seeds per energy without launching jobs.
2. How would you recognize accidental repeated events caused by restarting seeds?
3. What information would a colleague need to reproduce a figure after you move the simulation directory?

SOURCE / scripts/campaign.sh; Makefile; original macros/merge.C; CxRoot.cc:Run and rootOut.

Everyday use

Diagnose compilation

OBJECTIVE Use the first relevant error as evidence instead of making arbitrary changes.

9.4 Tool not found [documentation/code]

For root-config: command not found, gfortran: not found or make: not found, use command -v and check environment activation and versions. Changing physics parameters cannot fix a missing compiler. In Conda, inspect CC, CXX and FC and pass the wrappers to Make when needed.

9.5 Missing header or library [documentation/code]

TFile.h: No such file suggests unconfigured ROOT or a wrong include path. Missing bzlib.h or cannot find -lbz2 suggests incomplete bzip2 development files. Compare root-config –cflags –libs with the actual compile line in the log. Incompatible ROOT binaries and compilers can fail during linking or execution. Keep logs before rebuilding.

9.6 Legacy Fortran and recent compilers [documentation/code]

The Makefile uses -std=legacy, which already tolerates certain argument incompatibilities in GNU Fortran [13]. Do not suppress diagnostics without understanding them. Record each source/recipe patch, reason and compiler version, and repeat the small test before producing more events. This does not constitute broad physics validation.

9.7 Paths and installation [code]

The executable stores a prefix for locating lib/Linux. Copying only bin/conex2r can leave loading pointed at the previous installation. Spaces can break Make or Fortran param-file opening. Use a short path without spaces and recompile after prefix changes. There is a macOS library-extension discrepancy; that platform was not tested for this edition.

SOURCE / Makefile; ConexDynamicInterface.cc; GNU Fortran documentation [13]; external test logs.

Everyday use

Diagnose simulation and analysis

OBJECTIVE Distinguish installation failures, incomplete data and unsuitable observables.

Symptom	Checkable diagnostic
`can not open CONEX shared library`	Loader path, model library existence and ldd on it; a different model target is insufficient.
`InitConex: Error opening parameter file`	CONEX_ROOT, filename selected by -m, readability and spaces.
`Cannot open file` for tables	Exact printed filename, existence and readability; compare fconex/fegs/furqmd entries.
Incompatible table version	Match distribution and dataset; do not rename another model's table to pass the check.
Empty/incomplete ROOT file	Full log, exit status, READ opening, Header and Shower entry count.
Missing branch	ls and Print, spelling, OutputVersion, and Header versus Shower.
Reading fails or values are implausible	Leaf type, variable dimension, nX counter and optional branches.
Empty histogram	Entries, expression, selection, finite values and limits; check underflow/overflow.
Maximum at a boundary	Simulated interval, ground and profile on both sides; do not interpret fits outside their support.

9.8 A troubleshooting sequence

Reproduce with one event and recorded commands. Check directory and variables, then model, param file and tables; read the log, open the output and finally run the macro. Change one factor at a time. Keep both the initial diagnosis and the successful change. Do not conclude that ROOT is broken when the simulator never wrote the file.

9.9 Cost and resources

Total elapsed time includes initialization, table reads, simulation and writing. Per-event cpuTime is not wall-clock time. More events, greater -K detail, geometry or threshold changes and primary mass can increase runtime and disk use. Measure a small sample and estimate requirements before a large campaign. No universal time per shower is provided.

SOURCE / Messages in ConexDynamicInterface.cc, conex2r.F and Interface.bz2.cc; the supplied reading macros.

10 / REFERENCE
Guide / APOEMA

Linked glossary

Physics and tools in a few lines

OBJECTIVE Clarify terms used in the guide and return to their first occurrence.

Terms already explained in concept boxes remain defined in context. This glossary supplies complementary vocabulary, with return links that work inside the PDF.

10.1 Physics and geometry

Zenith angle. Angle between arrival direction and the local vertical. In this convention, 0° means vertically downward, 90° horizontal, and values above 90° upward-going showers.

Return to the text.

Extensive air showers. Particle cascades generated by atmospheric interactions and decays. They develop longitudinally and laterally; this guide uses the axial profiles calculated by CONEX.

Return to the text.

Upward-going shower. A shower propagating upward through the atmosphere. It corresponds to zenith angles above 90° in this distribution; selecting this direction does not enable unsupported primaries or processes.

Return to the text.

CORSIKA. A detailed particle-cascade simulation program tracking trajectories and interactions. CORSIKA and CONEX are distinct programs; integration options do not make their installation or execution instructions interchangeable.

Return to the text.

Cosmic ray. An extraterrestrial particle reaching the atmosphere; protons and nuclei are central examples here. The incident primary and atmospheric secondaries play different roles in interpreting a shower.

Return to the text.

SOURCE / README; hybrid method [1, 2]; distribution geometry and model conventions.

Linked glossary

Processes and the hybrid method

Hadronic interactions. Collisions involving hadrons or nuclei and the strong interaction, producing secondaries. CONEX combines high- and low-energy models; their choices affect multiplicities, energies and cascade development.

Return to the text.

Cascade equations. Transport equations describing particle populations in energy and depth, including production and losses. In the hybrid method, source terms from Monte Carlo feed the numerical lower-energy evolution.

Return to the text.

Monte Carlo. A method using random sampling to represent probabilistic processes. Hybrid CONEX simulates the higher-energy part this way; cascade equations complete evolution below the selected thresholds.

Return to the text.

Source terms. Contributions feeding an equation with particles from other processes. At the Monte Carlo/cascade-equation transition, they carry information about particles leaving explicit tracking into the numerical population evolution.

Return to the text.

Gaisser–Hillas. A function for fitting a longitudinal profile and estimating its maximum. Fit parameter X_0 is not automatically first-interaction depth; check fit quality against the profile.

Return to the text.

SOURCE / README; hybrid method [1, 2]; distribution geometry and model conventions.

Linked glossary

Tools and conventions

Bash. The shell and command language in the Linux examples. It interprets variables such as `$CX`, quotes, redirections and scripts; another shell can require different syntax.

Return to the text.

Paths. For `cfg/conex_sibyll.param`, lookup starts in the current directory, shown by pwd. After cd, the same relative path can identify a different location; .. denotes the parent directory.

Return to the text.

Flag. An option supplied to a program, often followed by a value, as in -n 1. The same letter can mean something else in another executable; runtime flags and compilation options act at different stages.

Return to the text.

Fortran. The programming language used for CONEX physics routines and included models. Sources require a Fortran compiler; this distribution's .F files also undergo preprocessing to select code sections.

Return to the text.

Macros. In ROOT, C++ files executed to read data or draw plots. Compilation macros, by contrast, are preprocessor symbols selecting source code; context determines the meaning.

Return to the text.

PowerShell. The shell and automation language in Windows-labelled blocks. Its commands and variables have their own conventions; opening Ubuntu in WSL changes the environment where Linux commands execute.

Return to the text.

WSL 2. A Windows Subsystem for Linux architecture running a Linux kernel in a lightweight virtual machine. CONEX there still follows the Linux route; it does not establish native Windows support.

Return to the text.

PDG. The Particle Data Group compiles particle properties and maintains a numerical identification convention used in simulations. CONEX nuclear input uses $100A$; detailed output can use the different CORSIKA convention.

Return to the text.

Wall-clock time. Actual elapsed time between the start and end of a task, as on a stopwatch. It includes processing and waits, such as reading and writing, and need not equal the CPU time recorded per event.

Return to the text.

SOURCE / Official Bash, GNU Fortran, ROOT, PowerShell and WSL documentation; PDG convention [18].

References and further reading

Primary sources and official documentation

OBJECTIVE Locate the fundamentals and check versions before updating the guide.

Start with Bergmann et al. for the method and Pierog et al. for its historical context. Model lists in early papers are not a catalogue of the 7.801 copy. The README requests both papers, prioritizing the first, and acknowledges the original Bossard et al. work. Those credits are preserved below.

References

[1] T. Bergmann, R. Engel, D. Heck, N. N. Kalmykov, S. Ostapchenko, T. Pierog, T. Thouw and K. Werner. *One-dimensional Hybrid Approach to Extensive Air Shower Simulation*. Astroparticle Physics 26 (2007), 420–432. DOI: 10.1016/j.astropartphys.2006.08.005. arXiv:astro-ph/0606564 (2006).

[2] T. Pierog et al. *First Results of Fast One-dimensional Hybrid Simulation of EAS Using CONEX*. Nuclear Physics B – Proceedings Supplements 151 (2006), 159–162. DOI: 10.1016/j.nuclphysbps.2005.07.029. arXiv:astro-ph/0411260 (2004).

[3] G. Bossard et al. *Cosmic ray air shower characteristics in the framework of the parton-based Gribov–Regge model NEXUS*. Physical Review D 63 (2001), 054030. arXiv:hep-ph/0009119. Original work acknowledged in the README.

[4] R. Ulrich, R. Engel and M. Unger. *Hadronic Multiparticle Production at Ultra-High Energies and Extensive Air Showers*. Physical Review D 83 (2011), 054026. DOI:10.1103/PhysRevD.83.054026. arXiv:1010.4310.

[5] cxroot/CONEX authors. Repository and official tags. Documentary comparison: conex2r7.60 and conex2r7.801. Accessed 23 October 2026.

[6] Air Shower Physics. corsika-data, institutional table repository; tree d8ac240e consulted as a reference for tag 7.801. Accessed 23 October 2026.

[7] KIT, CORSIKA. Downloads and access conditions. Do not confuse the CORSIKA version with the standalone CONEX executable version. Accessed 23 October 2026.

References and further reading

Tool documentation and local material

OBJECTIVE Keep reference links available without requiring them for offline reading.

References

[8] ROOT/CERN. Installing ROOT. Methods and activation; tested versions are recorded separately. Accessed 23 October 2026.

[9] ROOT/CERN. Dependencies. Requirements by platform and compiler. Accessed 23 October 2026.

[10] ROOT/CERN. TTree Class Reference. Tree structure and reading. Accessed 23 October 2026.

[11] ROOT/CERN. TFile Class Reference. Files, reading and persistence. Accessed 23 October 2026.

[12] ROOT/CERN. TH1 Class Reference. Histograms and bin conventions. Accessed 23 October 2026.

[13] GNU Project. GNU Fortran: Fortran Dialect Options; Argument Passing Conventions. Accessed 23 October 2026.

[14] Conda. Managing Environments. Accessed 23 October 2026. Tests used Micromamba and conda-forge packages.

[15] Microsoft Learn. How to install Linux on Windows with WSL. Accessed 23 October 2026.

[16] ROOT/CERN. Python Interface. Access to ROOT objects from Python. Accessed 25 October 2026.

[17] ROOT/CERN. TGraph Class Reference. Ordered pairs and plots. Accessed 25 October 2026.

[18] Particle Data Group. Monte Carlo Particle Numbering Scheme, 2026 revision. Accessed 25 October 2026.

11.1 Local sources and technical credits

Marcos Olegário: CONEX.md, the earlier manual; identity confirmed by the group.

Local distribution: README, Makefile, CMakeLists.txt, sources, paramin files, macros and auxiliary documentation were checked against the procedures. CONEX and ROOT-interface authors are acknowledged following the README; this guide does not attribute simulator authorship to APOEMA. In publications, also cite the models actually used according to their authors and distribution.

Result figures were generated from the tests; the hybrid-method diagram is original conceptual artwork. No external photographs or illustrations requiring reproduction licences were used. Supplied documents were treated as study material, not instructions to install programs or change the system.

12 / REFERENCE
Guide / APOEMA

Linux commands

Navigate, inspect files and find help

OBJECTIVE Quickly find the purpose of commands used throughout the guide.

These tables collect Linux/Bash commands and common navigation aids. Examples are for reference: adapt filenames and paths before running them. Windows commands and ROOT-interpreter calls are distinguished at the end.

Command or example	Purpose in this guide
`pwd`	Show the current directory; interpret relative paths.
`ls`	List the current directory.
`ls -lh first-run`	Show details, permissions and human-readable sizes.
`cd "$CX"`	Enter the directory stored in CX.
`cd ..`	Move to the parent directory.
`cd`	Return to the Linux user's home.
`mkdir first-run`	Create the directory; without -p, report if it already exists.
`cp file copy`	Copy a file, retaining the original; cp -r copies directories.
`mv old new`	Rename or move a file/directory. Check the destination to preserve data.
`rm -i file`	Remove a file after confirmation; useful when organizing results.
`cat file`	Display text; used here to identify the Linux distribution.
`head -n 20 file`	Display the first 20 lines, as for a param file.
`tail -n 25 run.log`	Display the log's final 25 lines.
`less run.log`	Browse text; q quits. Useful for long logs.
`man ls` / `help cd`	Separate calls: open the ls manual or Bash's builtin cd help.

SOURCE / GNU Coreutils and Bash Reference Manual.

Linux commands

Check the system and prepare tools

OBJECTIVE Look up diagnostics, installation and activation commands.

Command or example	Purpose in this guide
`uname -a`	Kernel, system and architecture information.
`command -v make`	Show how the shell locates Make; accepts multiple tool names.
`gcc -version`	C compiler version; g++ and gfortran provide the C++ and Fortran equivalents.
`"$CXX" -version`	Run the compiler identified by CXX; use `"$FC" -version` for Fortran.
`root-config -version`	Version of ROOT in the current session.
`root-config -cflags -libs`	Compiler and linker flags required by this ROOT installation.
`ldd library.so`	Library dependencies for investigating loading.
`sudo apt update`	Refresh Ubuntu/Debian package indexes with administrative privileges.
`sudo apt install packages`	Install the indicated system dependencies.
`bash file.sh`	Execute a Bash script, including installer and example scripts.
`source file.sh`	Read instructions in the current shell to prepare tools or Conda.
`conda create -n conex-apoema`	Create the separate working environment.
`conda activate conex-apoema`	Activate it; conda deactivate leaves it.
`conda install -c conda-forge root`	Install ROOT from conda-forge in the active environment.
`conda config -env ...`	Change environment settings; this guide sets channel_priority strict.
`conda -version`	Check manager availability and version.

SOURCE / Bash and Coreutils; official Conda, ROOT and Ubuntu documentation.

Linux commands

Compile, run and record results

OBJECTIVE Distinguish programs launched from the terminal from shell features.

Command or example	Purpose in this guide
`make sibyll`	Build the Makefile's SIBYLL target; make with no target displays this package's help.
`make all / make clean`	Separate calls: prepare all models or remove build products. Unnecessary for repeating the first run.
`bin/conex2r -h`	Show help from the compiled CONEX executable.
`bin/conex2r [options]`	Run with selected flags; see section 6.
`root -l file.root`	Open ROOT with a file for interactive exploration.
`root -l -b -q macro.C`	Execute without windows and quit; argument calls need the quoting shown in the examples.
`python -c "import ROOT"`	Check Python access to PyROOT.
`python python/conex.py -help`	Show the supplied Python analysis operations.
`export ROOT_OUT="directory"`	Set and pass a variable to programs launched from this session.
`printf`	Print formatted text, including a stored exit status.
`tee file.log`	Display and save incoming text; preserve simulator exit status in pipelines.
`set -euo pipefail`	Configure Bash failure handling in the supplied scripts.

SOURCE / CONEX README and Makefile; Bash Reference Manual; official ROOT and pdfLaTeX documentation.

Linux commands

Read syntax and distinguish contexts

OBJECTIVE Recognize shell symbols and calls belonging to other programs.

Syntax	Purpose in this guide
`CX="path"`	Shell assignment. Do not insert spaces around =.
`$HOME / $CX`	Variable expansions: the user's home and the path chosen here.
`"$CX"`	Double quotes preserve one argument, even if the value contains spaces.
`status=$?`	Store the previous command's status immediately after it finishes.
`> file.log`	Redirect standard output, replacing earlier file contents.
`2>&1`	Send standard error to standard output's already configured destination.
`|`	Connect one command's output to another's input, as with tee.
`\`	At a line's end, continue the same command on the next line. Do not put spaces or text after it.
`# comment`	Begin a comment outside quotes; not part of the executed command.

12.1 Quick run checklist

1. Active environment: command -v root-config and compiler versions.
2. Run root: `cd "$CX"`; configuration: `CONEX_ROOT="$CX/cfg"`.
3. New output directory; never rely on the prefix alone to protect results.
4. Compiled model and corresponding tables available.
5. Both energy limits and both zenith limits explicitly chosen.
6. Positive seed, log and manifest recorded.
7. Check output, Header, entry counts and schema before plotting.

12.2 Windows and the ROOT interpreter

`wsl -install -d Ubuntu`, `wsl -list -verbose` and `wsl -d Ubuntu` belong to PowerShell/Windows. Calls such as ls(), Print(), Show(), Draw() and .q belong to ROOT in these examples, not directly to the Bash terminal.

SOURCE / Bash Reference Manual; Microsoft/WSL; ROOT references in section 7.